

22. How do you implement and evaluate RAG (Retrieval-Augmented Generation) systems?

Answer:

RAG System Architecture:

1. Core Components:

```
python
```

```

from sentence_transformers import SentenceTransformer
import faiss
import numpy as np
from transformers import AutoTokenizer, AutoModelForCausalLM

class RAGSystem:
    def __init__(self, retriever_model, generator_model, vector_db_path):
        # Retriever component
        self.retriever = SentenceTransformer(retriever_model)
        self.vector_db = faiss.read_index(vector_db_path)

        # Generator component
        self.tokenizer = AutoTokenizer.from_pretrained(generator_model)
        self.generator = AutoModelForCausalLM.from_pretrained(generator_model)

        # Document store
        self.documents = self.load_documents()

    def retrieve(self, query, k=5):
        """Retrieve relevant documents for a query"""
        # Encode query
        query_embedding = self.retriever.encode([query])

        # Search in vector database
        scores, indices = self.vector_db.search(query_embedding, k)

        # Return documents with scores
        retrieved_docs = []
        for i, (score, idx) in enumerate(zip(scores[0], indices[0])):
            retrieved_docs.append({
                'text': self.documents[idx],
                'score': float(score),
                'rank': i + 1
            })

        return retrieved_docs

    def generate_response(self, query, retrieved_docs, max_length=512):
        """Generate response using query and retrieved documents"""

        # Construct prompt with retrieved context
        context = self.format_context(retrieved_docs)
        prompt = self.build_prompt(query, context)

        # Tokenize and generate
        inputs = self.tokenizer(prompt, return_tensors="pt", truncation=True, max_length=1024)

```

```

with torch.no_grad():
    outputs = self.generator.generate(
        **inputs,
        max_length=max_length,
        num_return_sequences=1,
        temperature=0.7,
        do_sample=True,
        pad_token_id=self.tokenizer.eos_token_id
    )

    # Decode response
    response = self.tokenizer.decode(outputs[0], skip_special_tokens=True)
    response = response[len(prompt):].strip() # Remove prompt from response

    return response

```

```

def format_context(self, retrieved_docs, max_docs=3):
    """Format retrieved documents as context"""
    context_parts = []
    for i, doc in enumerate(retrieved_docs[:max_docs]):
        context_parts.append(f"Document {i+1}: {doc['text']}")
    return "\n\n".join(context_parts)

def build_prompt(self, query, context):
    """Build prompt for generation"""
    return f"Based on the following context, answer the question:

```

Context:
{context}

Question: {query}

Answer:""

```

def answer(self, query):
    """End-to-end RAG pipeline"""
    # Retrieve relevant documents
    retrieved_docs = self.retrieve(query)

    # Generate response
    response = self.generate_response(query, retrieved_docs)

    return {
        'query': query,
        'response': response,
        'retrieved_documents': retrieved_docs,
    }

```

```
'sources': [doc['text'][:100] + '...' for doc in retrieved_docs[:3]]
}
```

2. Advanced Retrieval Strategies:

Hybrid Retrieval (Dense + Sparse):

```
python
```

```
from rank_bm25 import BM25Okapi
```

```
class HybridRetriever:
```

```
    def __init__(self, dense_model, documents):
```

```
        self.dense_retriever = SentenceTransformer(dense_model)
```

```
        self.documents = documents
```

```
        # BM25 for sparse retrieval
```

```
        tokenized_docs = [doc.split() for doc in documents]
```

```
        self.bm25 = BM25Okapi(tokenized_docs)
```

```
        # Pre-compute dense embeddings
```

```
        self.doc_embeddings = self.dense_retriever.encode(documents)
```

```
        self.dense_index = faiss.IndexFlatIP(self.doc_embeddings.shape[1])
```

```
        self.dense_index.add(self.doc_embeddings.astype('float32'))
```

```
    def hybrid_retrieve(self, query, k=10, alpha=0.5):
```

```
        """Combine dense and sparse retrieval"""
```

```
        # Dense retrieval
```

```
        query_embedding = self.dense_retriever.encode([query])
```

```
        dense_scores, dense_indices = self.dense_index.search(
```

```
            query_embedding.astype('float32'), k
```

```
        )
```

```
        # Sparse retrieval (BM25)
```

```
        tokenized_query = query.split()
```

```
        bm25_scores = self.bm25.get_scores(tokenized_query)
```

```
        sparse_indices = np.argsort(bm25_scores)[::-1][:k]
```

```
        # Combine scores
```

```
        combined_scores = {}
```

```
        # Add dense scores
```

```
        for score, idx in zip(dense_scores[0], dense_indices[0]):
```

```
            combined_scores[idx] = alpha * score
```

```
        # Add sparse scores
```

```
        for idx in sparse_indices:
```

```
            if idx in combined_scores:
```

```
                combined_scores[idx] += (1 - alpha) * bm25_scores[idx]
```

```
            else:
```

```
                combined_scores[idx] = (1 - alpha) * bm25_scores[idx]
```

```
        # Sort by combined score
```

```
        sorted_results = sorted(combined_scores.items(), key=lambda x: x[1], reverse=True)
```

```
return [(idx, score) for idx, score in sorted_results[:k]]
```

Multi-hop Retrieval:

```
python
```

```

class MultiHopRAG:
    def __init__(self, retriever, generator):
        self.retriever = retriever
        self.generator = generator
        self.max_hops = 3

    def multi_hop_retrieve(self, query):
        """Perform multi-hop retrieval for complex queries"""
        all_retrieved = []
        current_query = query

        for hop in range(self.max_hops):
            # Retrieve documents for current query
            docs = self.retriever.retrieve(current_query, k=5)
            all_retrieved.extend(docs)

            # Generate follow-up query if needed
            if hop < self.max_hops - 1:
                follow_up_query = self.generate_follow_up_query(current_query, docs)
                if follow_up_query and follow_up_query != current_query:
                    current_query = follow_up_query
            else:
                break

        # Remove duplicates and re-rank
        unique_docs = self.deduplicate_and_rerank(all_retrieved, query)
        return unique_docs

    def generate_follow_up_query(self, original_query, retrieved_docs):
        """Generate follow-up query based on retrieved documents"""
        context = "\n".join([doc['text'][:200] for doc in retrieved_docs[:2]])

        prompt = f"""Given the original query and some retrieved information,
        generate a follow-up query to gather more relevant information.

        Original query: {original_query}
        Retrieved information: {context}

        Follow-up query: """

        # Use generator to create follow-up query
        follow_up = self.generator.generate_response(prompt, [])
        return follow_up.strip()

```

3. Evaluation Metrics:

Retrieval Evaluation:

python

```

class RAGEvaluator:
    def __init__(self):
        self.retrieval_metrics = []
        self.generation_metrics = []

    def evaluate_retrieval(self, queries, ground_truth_docs, retrieved_docs):
        """Evaluate retrieval component"""
        results = {}

        # Recall@K
        def recall_at_k(retrieved, relevant, k):
            retrieved_k = set(retrieved[:k])
            relevant_set = set(relevant)
            if not relevant_set:
                return 0.0
            return len(retrieved_k.intersection(relevant_set)) / len(relevant_set)

        # Precision@K
        def precision_at_k(retrieved, relevant, k):
            retrieved_k = set(retrieved[:k])
            relevant_set = set(relevant)
            if not retrieved_k:
                return 0.0
            return len(retrieved_k.intersection(relevant_set)) / len(retrieved_k)

        # Mean Reciprocal Rank
        def mrr(retrieved, relevant):
            for i, doc in enumerate(retrieved):
                if doc in relevant:
                    return 1.0 / (i + 1)
            return 0.0

        # Calculate metrics for each query
        recall_scores = []
        precision_scores = []
        mrr_scores = []

        for query_idx, (retrieved, relevant) in enumerate(zip(retrieved_docs, ground_truth_docs)):
            recall_scores.append(recall_at_k(retrieved, relevant, k=5))
            precision_scores.append(precision_at_k(retrieved, relevant, k=5))
            mrr_scores.append(mrr(retrieved, relevant))

        results = {
            'recall@5': np.mean(recall_scores),
            'precision@5': np.mean(precision_scores),
            'mrr': np.mean(mrr_scores)
        }

```

```
}
```

```
return results
```

```
def evaluate_generation(self, generated_responses, reference_answers):
```

```
    """Evaluate generation quality"""
```

```
    from rouge_score import rouge_scorer
```

```
    from bert_score import score
```

```
    # ROUGE scores
```

```
    rouge_scorer_obj = rouge_scorer.RougeScorer(['rouge1', 'rouge2', 'rougeL'])
```

```
    rouge_scores = {'rouge1': [], 'rouge2': [], 'rougeL': []}
```

```
    for gen, ref in zip(generated_responses, reference_answers):
```

```
        scores = rouge_scorer_obj.score(ref, gen)
```

```
        for metric in rouge_scores:
```

```
            rouge_scores[metric].append(scores[metric].fmeasure)
```

```
    # BERTScore
```

```
    P, R, F1 = score(generated_responses, reference_answers, lang='en')
```

```
    results = {
```

```
        'rouge1': np.mean(rouge_scores['rouge1']),
```

```
        'rouge2': np.mean(rouge_scores['rouge2']),
```

```
        'rougeL': np.mean(rouge_scores['rougeL']),
```

```
        'bertscore_f1': F1.mean().item()
```

```
    }
```

```
    return results
```

```
def evaluate_faithfulness(self, generated_responses, retrieved_docs):
```

```
    """Evaluate if generated responses are faithful to retrieved documents"""
```

```
    faithfulness_scores = []
```

```
    for response, docs in zip(generated_responses, retrieved_docs):
```

```
        # Use NLI model to check entailment
```

```
        context = " ".join([doc['text'] for doc in docs[:3]])
```

```
        entailment_score = self.check_entailment(response, context)
```

```
        faithfulness_scores.append(entailment_score)
```

```
    return {'faithfulness': np.mean(faithfulness_scores)}
```

```
def check_entailment(self, response, context):
```

```
    """Check if response is entailed by context using NLI model"""
```

```
    # Simplified - in practice, use a trained NLI model
```

```
    from transformers import pipeline
```

```

nli_model = pipeline("text-classification",
                      model="microsoft/DialoGPT-medium")

result = nli_model(f'{context} [SEP] {response}')
entailment_score = max([r['score'] for r in result if r['label'] == 'ENTAILMENT'])
return entailment_score

```

4. Optimization Techniques:

Query Rewriting:

```

python

class QueryRewriter:
    def __init__(self, rewriter_model):
        self.rewriter = pipeline("text2text-generation", model=rewriter_model)

    def rewrite_query(self, query, retrieval_results=None):
        """Rewrite query for better retrieval"""

        if retrieval_results and len(retrieval_results) > 0:
            # Context-aware rewriting
            context = retrieval_results[0]['text'][:100]
            prompt = f"Rewrite this query for better search results. Context: {context}\nQuery: {query}\nRewritten:"
        else:
            # Simple query expansion
            prompt = f"Expand this query with relevant terms: {query}\nExpanded:"

        rewritten = self.rewriter(prompt, max_length=50, do_sample=True)
        return rewritten[0]['generated_text']

```

Re-ranking:

```
class CrossEncoderReranker:
    def __init__(self, reranker_model):
        from sentence_transformers import CrossEncoder
        self.reranker = CrossEncoder(reranker_model)

    def rerank(self, query, retrieved_docs, top_k=5):
        """Re-rank retrieved documents using cross-encoder"""

        # Create query-document pairs
        pairs = [(query, doc['text']) for doc in retrieved_docs]

        # Score pairs
        scores = self.reranker.predict(pairs)

        # Re-rank based on scores
        scored_docs = [(doc, score) for doc, score in zip(retrieved_docs, scores)]
        scored_docs.sort(key=lambda x: x[1], reverse=True)

        return [doc for doc, _ in scored_docs[:top_k]]
```

5. Production Considerations:

Caching and Performance:

```
python
```

```

import redis
import pickle

class CachedRAGSystem(RAGSystem):
    def __init__(self, *args, **kwargs):
        super().__init__(*args, **kwargs)
        self.cache = redis.Redis(host='localhost', port=6379, db=0)
        self.cache_ttl = 3600 # 1 hour

    def retrieve_with_cache(self, query, k=5):
        """Retrieve with caching"""
        cache_key = f"retrieve:{hash(query)}:{k}"

        # Check cache
        cached_result = self.cache.get(cache_key)
        if cached_result:
            return pickle.loads(cached_result)

        # Retrieve and cache
        result = self.retrieve(query, k)
        self.cache.setex(cache_key, self.cache_ttl, pickle.dumps(result))

        return result

    def async_index_update(self, new_documents):
        """Asynchronously update vector index"""
        # Implementation for updating index without downtime
        pass

```

Monitoring and Logging:

```
python
```

```

import logging
from prometheus_client import Counter, Histogram

class MonitoredRAGSystem(RAGSystem):
    def __init__(self, *args, **kwargs):
        super().__init__(*args, **kwargs)

        # Metrics
        self.query_counter = Counter('rag_queries_total', 'Total RAG queries')
        self.retrieval_latency = Histogram('rag_retrieval_duration_seconds', 'Retrieval latency')
        self.generation_latency = Histogram('rag_generation_duration_seconds', 'Generation latency')

        # Logging
        logging.basicConfig(level=logging.INFO)
        self.logger = logging.getLogger(__name__)

    def answer(self, query):
        self.query_counter.inc()
        self.logger.info(f"Processing query: {query[:50]}...")

        start_time = time.time()

        try:
            result = super().answer(query)

            # Log success metrics
            self.logger.info(f"Query processed successfully in {time.time() - start_time:.2f}s")

            return result

        except Exception as e:
            self.logger.error(f"Error processing query: {e}")
            raise

```

Best Practices:

1. **Document Preprocessing:** Clean, chunk, and enrich documents before indexing
2. **Embedding Models:** Use domain-specific embeddings when possible
3. **Chunk Strategy:** Experiment with different chunking strategies (sentence, paragraph, semantic)
4. **Context Window Management:** Ensure retrieved context fits within model's context window
5. **Citation Generation:** Track and provide source citations for generated responses
6. **Continuous Evaluation:** Monitor retrieval and generation quality in production
7. **A/B Testing:** Test different retrieval strategies and generation approaches

Explanation: RAG systems combine the strengths of retrieval and generation, enabling factual, up-to-date responses while maintaining the fluency of large language models. Success depends on optimizing both retrieval relevance and generation quality while maintaining system performance and reliability.

23. What are the key techniques for prompt engineering and few-shot learning?

Answer:

Prompt Engineering Fundamentals:

1. Basic Prompt Structure:

```
python

class PromptTemplate:
    def __init__(self, template, input_variables):
        self.template = template
        self.input_variables = input_variables

    def format(self, **kwargs):
        """Format template with provided variables"""
        return self.template.format(**kwargs)

# Example templates
classification_template = PromptTemplate(
    template="""
    Classify the sentiment of the following text as positive, negative, or neutral.

    Text: {text}
    Sentiment: """,
    input_variables=["text"]
)

reasoning_template = PromptTemplate(
    template="""
    Let's think step by step to solve this problem.

    Problem: {problem}

    Step 1: """,
    input_variables=["problem"]
)
```

2. Few-Shot Learning Patterns:

In-Context Learning:

python

```

class FewShotPromptBuilder:
    def __init__(self, task_description, examples, input_format, output_format):
        self.task_description = task_description
        self.examples = examples
        self.input_format = input_format
        self.output_format = output_format

    def build_prompt(self, new_input):
        """Build few-shot prompt with examples"""

        prompt_parts = [self.task_description, ""]

        # Add examples
        for i, example in enumerate(self.examples):
            prompt_parts.append(f"Example {i+1}:")
            prompt_parts.append(self.input_format.format(**example['input']))
            prompt_parts.append(self.output_format.format(**example['output']))
            prompt_parts.append("")

        # Add new input
        prompt_parts.append("Now solve:")
        prompt_parts.append(self.input_format.format(**new_input))
        prompt_parts.append(self.output_format.format(**{"output": ""}))

        return "\n".join(prompt_parts)

# Example usage for text classification
few_shot_classifier = FewShotPromptBuilder(
    task_description="Classify the sentiment of movie reviews.",
    examples=[
        {
            "input": {"review": "This movie was absolutely amazing! Great acting and plot."},
            "output": {"sentiment": "Positive"}
        },
        {
            "input": {"review": "Terrible movie, waste of time and money."},
            "output": {"sentiment": "Negative"}
        },
        {
            "input": {"review": "The movie was okay, nothing special but watchable."},
            "output": {"sentiment": "Neutral"}
        }
    ],
    input_format="Review: {review}",
    output_format="Sentiment: {sentiment}"
)

```

```
# Generate prompt for new input
```

```
new_review = {"review": "The cinematography was stunning but the story felt rushed."}
```

```
prompt = few_shot_classifier.build_prompt(new_review)
```

3. Advanced Prompting Techniques:

Chain-of-Thought Prompting:

```
python
```

```
def create_chain_of_thought_prompt(problem, examples_with_reasoning):
```

```
    """Create prompt that encourages step-by-step reasoning"""
```

```
    prompt = "Solve these problems step by step, showing your reasoning.\n\n"
```

```
    # Add examples with reasoning
```

```
    for i, example in enumerate(examples_with_reasoning):
```

```
        prompt += f"Problem {i+1}: {example['problem']}\n"
```

```
        prompt += f"Reasoning: {example['reasoning']}\n"
```

```
        prompt += f"Answer: {example['answer']}\n\n"
```

```
    # Add new problem
```

```
    prompt += f"Problem: {problem}\n"
```

```
    prompt += "Reasoning: Let me think step by step.\n"
```

```
    return prompt
```

```
# Example for math problems
```

```
math_examples = [
```

```
    {
        "problem": "A store has 15 apples. They sell 8 apples and then buy 12 more. How many apples do they have now?"
        "reasoning": "Starting with 15 apples, after selling 8:  $15 - 8 = 7$  apples. After buying 12 more:  $7 + 12 = 19$  apples.",
        "answer": "19 apples"
    }
```

```
]
```

```
cot_prompt = create_chain_of_thought_prompt(
```

```
    "A library has 45 books. Students borrow 23 books and return 8 books. How many books are in the library now?",
    math_examples
```

```
)
```

Self-Consistency Prompting:

```
python
```

```

import random
from collections import Counter

class SelfConsistencyPrompting:
    def __init__(self, model, num_samples=5):
        self.model = model
        self.num_samples = num_samples

    def generate_multiple_responses(self, prompt, temperature=0.7):
        """Generate multiple responses for the same prompt"""
        responses = []

        for _ in range(self.num_samples):
            response = self.model.generate(
                prompt,
                temperature=temperature,
                max_tokens=150,
                stop=["\n\n"]
            )
            responses.append(response.strip())

        return responses

    def extract_answer(self, response):
        """Extract final answer from response"""
        # Simple extraction - look for "Answer:" or number at the end
        lines = response.split('\n')
        for line in reversed(lines):
            if 'Answer:' in line:
                return line.split('Answer:')[1].strip()
            # Look for number patterns
            import re
            numbers = re.findall(r'\d+', line)
            if numbers:
                return numbers[-1]
        return response.split()[-1] if response.split() else ""

    def get_consensus_answer(self, prompt):
        """Get consensus answer using self-consistency"""
        responses = self.generate_multiple_responses(prompt)
        answers = [self.extract_answer(response) for response in responses]

        # Find most common answer
        answer_counts = Counter(answers)
        consensus_answer = answer_counts.most_common(1)[0][0]
        confidence = answer_counts[consensus_answer] / len(answers)

```

```

return {
    'consensus_answer': consensus_answer,
    'confidence': confidence,
    'all_responses': responses,
    'all_answers': answers
}

```

4. Task-Specific Prompting Strategies:

Code Generation:

```

python

class CodeGenerationPrompts:
    @staticmethod
    def function_generation_prompt(description, examples=None, language="Python"):
        """Generate prompt for function creation"""
        prompt = f"Write a {language} function that {description}.\n\n"

        if examples:
            prompt += "Examples:\n"
            for example in examples:
                prompt += f"Input: {example['input']}\n"
                prompt += f"Output: {example['output']}\n\n"

        prompt += f"{language} function:\n```${language.lower()}\n"
        return prompt

    @staticmethod
    def code_explanation_prompt(code, language="Python"):
        """Generate prompt for code explanation"""
        return f"""Explain what this {language} code does step by step:

```${language.lower()}
{code}

```

### Explanation:

1. """

```

@staticmethod
def code_debugging_prompt(code, error, language="Python"):
 """Generate prompt for debugging"""
 return f"""Debug this {language} code that produces an error:

```

### Code:

```
{code}
```

Error: {error}

The issue is: ""

```
Data Analysis:
```

```
```python
```

```
class DataAnalysisPrompts:
```

```
    @staticmethod
```

```
    def data_description_prompt(data_sample, context=""):
```

```
        """Generate prompt for data analysis"""
```

```
        return f"""Analyze this dataset and provide insights:
```

```
Context: {context}
```

```
Data sample:
```

```
{data_sample}
```

```
Analysis:
```

```
1. Data structure and types:
```

```
2. Key patterns observed:
```

```
3. Potential insights:
```

```
4. Recommended next steps: ""
```

```
    @staticmethod
```

```
    def visualization_prompt(data_description, question):
```

```
        """Generate prompt for data visualization suggestions"""
```

```
        return f"""Given this data description and question, suggest the best visualization:
```

```
Data: {data_description}
```

```
Question: {question}
```

```
Recommended visualization:
```

```
1. Chart type:
```

```
2. X-axis:
```

```
3. Y-axis:
```

```
4. Additional parameters:
```

```
5. Why this visualization works best: ""
```

5. Dynamic Prompt Generation:

Context-Aware Prompting:

python

```

class AdaptivePromptGenerator:
    def __init__(self, base_examples, similarity_threshold=0.8):
        self.base_examples = base_examples
        self.similarity_threshold = similarity_threshold
        self.encoder = SentenceTransformer('all-MiniLM-L6-v2')

    def select_relevant_examples(self, query, max_examples=3):
        """Select most relevant examples for the query"""
        query_embedding = self.encoder.encode([query])

        example_similarities = []
        for example in self.base_examples:
            example_text = example['input']
            example_embedding = self.encoder.encode([example_text])

            # Compute cosine similarity
            similarity = np.dot(query_embedding[0], example_embedding[0]) / (
                np.linalg.norm(query_embedding[0]) * np.linalg.norm(example_embedding[0])
            )

            example_similarities.append((example, similarity))

        # Sort by similarity and select top examples
        example_similarities.sort(key=lambda x: x[1], reverse=True)
        selected = [ex for ex, sim in example_similarities[:max_examples]
                     if sim > self.similarity_threshold]

        return selected if selected else self.base_examples[:max_examples]

    def generate_adaptive_prompt(self, query, task_description):
        """Generate prompt with most relevant examples"""
        relevant_examples = self.select_relevant_examples(query)

        prompt_parts = [task_description, ""]

        # Add relevant examples
        for i, example in enumerate(relevant_examples):
            prompt_parts.append(f"Example {i+1}:")
            prompt_parts.append(f"Input: {example['input']}")
            prompt_parts.append(f"Output: {example['output']}")
            prompt_parts.append("")

        # Add current query
        prompt_parts.append(f"Input: {query}")
        prompt_parts.append("Output:")

```

```
return "\n".join(prompt_parts)
```

6. Prompt Optimization Techniques:

Automated Prompt Engineering:

```
python
```

```

class PromptOptimizer:
    def __init__(self, model, eval_dataset, metric_function):
        self.model = model
        self.eval_dataset = eval_dataset
        self.metric_function = metric_function

    def generate_prompt_variants(self, base_prompt):
        """Generate variations of a prompt"""
        variations = []

        # Different instruction phrasings
        instruction_variants = [
            "Please",
            "Your task is to",
            "I need you to",
            "Can you",
            "You should"
        ]

        # Different output format instructions
        format_variants = [
            "Format your response as:",
            "Please respond in the following format:",
            "Your answer should be:",
            "Provide your response as:"
        ]

        # Generate combinations
        for instruction in instruction_variants:
            for format_instr in format_variants:
                variant = base_prompt.replace("Classify", f"{instruction} classify")
                variant = variant.replace("Answer:", f"{format_instr} Answer:")
                variations.append(variant)

        return variations

    def evaluate_prompt(self, prompt):
        """Evaluate prompt performance on dataset"""
        predictions = []
        actuals = []

        for item in self.eval_dataset:
            formatted_prompt = prompt.format(**item['input'])
            prediction = self.model.generate(formatted_prompt)
            predictions.append(prediction)
            actuals.append(item['output'])

```

```
return self.metric_function(predictions, actuals)
```

```
def optimize_prompt(self, base_prompt, max_iterations=10):
```

```
    """Find best prompt variation"""
```

```
    best_prompt = base_prompt
```

```
    best_score = self.evaluate_prompt(base_prompt)
```

```
    for iteration in range(max_iterations):
```

```
        variants = self.generate_prompt_variants(best_prompt)
```

```
        for variant in variants:
```

```
            score = self.evaluate_prompt(variant)
```

```
            if score > best_score:
```

```
                best_score = score
```

```
                best_prompt = variant
```

```
    return best_prompt, best_score
```

7. Evaluation and Testing:

Prompt Performance Evaluation:

python

```

class PromptEvaluator:
    def __init__(self):
        self.metrics = {
            'accuracy': self.calculate_accuracy,
            'consistency': self.calculate_consistency,
            'robustness': self.calculate_robustness
        }

    def calculate_accuracy(self, predictions, ground_truth):
        """Calculate exact match accuracy"""
        correct = sum(pred.strip().lower() == gt.strip().lower()
                       for pred, gt in zip(predictions, ground_truth))
        return correct / len(predictions)

    def calculate_consistency(self, model, prompts, inputs, num_runs=5):
        """Measure consistency across multiple runs"""
        consistency_scores = []

        for prompt, input_data in zip(prompts, inputs):
            responses = []
            for _ in range(num_runs):
                response = model.generate(prompt.format(**input_data))
                responses.append(response.strip())

            # Calculate consistency as fraction of identical responses
            most_common = max(set(responses), key=responses.count)
            consistency = responses.count(most_common) / len(responses)
            consistency_scores.append(consistency)

        return np.mean(consistency_scores)

    def calculate_robustness(self, model, base_prompts, perturbed_prompts, inputs):
        """Measure robustness to prompt variations"""
        robustness_scores = []

        for base, perturbed, input_data in zip(base_prompts, perturbed_prompts, inputs):
            base_response = model.generate(base.format(**input_data))
            perturbed_response = model.generate(perturbed.format(**input_data))

            # Measure semantic similarity of responses
            similarity = self.semantic_similarity(base_response, perturbed_response)
            robustness_scores.append(similarity)

        return np.mean(robustness_scores)

    def semantic_similarity(self, text1, text2):

```

```
"""Calculate semantic similarity between two texts"""
```

```
# Simplified - use sentence transformers for better similarity
```

```
encoder = SentenceTransformer('all-MiniLM-L6-v2')
```

```
embeddings = encoder.encode([text1, text2])
```

```
similarity = np.dot(embeddings[0], embeddings[1]) / (
```

```
    np.linalg.norm(embeddings[0]) * np.linalg.norm(embeddings[1])
```

```
)
```

```
return similarity
```

Best Practices for Prompt Engineering:

1. **Clear Instructions:** Be explicit about the desired output format
2. **Relevant Examples:** Choose examples that closely match the target task
3. **Consistent Formatting:** Maintain consistent structure across examples
4. **Iterative Refinement:** Test and refine prompts based on performance
5. **Context Management:** Ensure prompts fit within model context limits
6. **Bias Awareness:** Check for and mitigate potential biases in examples
7. **Evaluation:** Use systematic evaluation with proper metrics
8. **Documentation:** Keep track of prompt versions and performance

Common Pitfalls to Avoid:

- Using ambiguous instructions
- Providing conflicting examples
- Over-fitting to specific examples
- Ignoring edge cases
- Not testing prompt robustness
- Assuming one-size-fits-all prompts

Explanation: Effective prompt engineering combines understanding of model capabilities with systematic experimentation and evaluation. The key is finding the right balance between providing sufficient context and maintaining clarity, while continuously optimizing based on performance metrics.

Agentic AI

24. What are the key components and architectures of AI agents?

Answer:

AI Agent Architecture Fundamentals:

1. Core Components:

python

```
from abc import ABC, abstractmethod
from dataclasses import dataclass
from typing import Any, Dict, List, Optional
import asyncio
```

```
@dataclass
```

```
class AgentState:
```

```
    """Represents the current state of an agent"""
```

```
    goals: List[str]
```

```
    beliefs: Dict[str, Any]
```

```
    memory: Dict[str, Any]
```

```
    current_task: Optional[str] = None
```

```
    context: Dict[str, Any] = None
```

```
@dataclass
```

```
class Action:
```

```
    """Represents an action that an agent can take"""
```

```
    name: str
```

```
    parameters: Dict[str, Any]
```

```
    expected_outcome: str
```

```
    confidence: float
```

```
class Agent(ABC):
```

```
    """Abstract base class for AI agents"""
```

```
    def __init__(self, agent_id: str, initial_state: AgentState):
```

```
        self.agent_id = agent_id
```

```
        self.state = initial_state
```

```
        self.action_history = []
```

```
        self.performance_metrics = {}
```

```
@abstractmethod
```

```
async def perceive(self, environment: Dict[str, Any]) -> Dict[str, Any]:
```

```
    """Perceive the current environment state"""
```

```
    pass
```

```
@abstractmethod
```

```
async def reason(self, observations: Dict[str, Any]) -> List[Action]:
```

```
    """Reason about possible actions given observations"""
```

```
    pass
```

```
@abstractmethod
```

```
async def act(self, action: Action) -> Dict[str, Any]:
```

```
    """Execute an action in the environment"""
```

```
    pass
```

```

async def run_cycle(self, environment: Dict[str, Any]) -> Dict[str, Any]:
    """Execute one complete agent cycle"""
    # Perceive
    observations = await self.perceive(environment)

    # Update beliefs based on observations
    self.update_beliefs(observations)

    # Reason about actions
    possible_actions = await self.reason(observations)

    # Select best action
    selected_action = self.select_action(possible_actions)

    # Execute action
    result = await self.act(selected_action)

    # Update state and memory
    self.update_state(selected_action, result)

    return result

def update_beliefs(self, observations: Dict[str, Any]):
    """Update agent beliefs based on new observations"""
    for key, value in observations.items():
        self.state.beliefs[key] = value

def select_action(self, actions: List[Action]) -> Action:
    """Select the best action from available options"""
    if not actions:
        return Action("wait", {}, "No action taken", 0.0)

    # Select action with highest confidence
    return max(actions, key=lambda a: a.confidence)

def update_state(self, action: Action, result: Dict[str, Any]):
    """Update agent state after action execution"""
    self.action_history.append((action, result))
    self.state.memory[f"action_{len(self.action_history)}"] = {
        'action': action,
        'result': result,
        'timestamp': asyncio.get_event_loop().time()
    }

```

2. Reactive Agents:

python

```

class ReactiveAgent(Agent):
    """Agent that responds directly to stimuli without complex reasoning"""

    def __init__(self, agent_id: str, initial_state: AgentState, rules: Dict[str, Action]):
        super().__init__(agent_id, initial_state)
        self.rules = rules # condition -> action mappings

    async def perceive(self, environment: Dict[str, Any]) -> Dict[str, Any]:
        """Simple perception - direct mapping of environment"""
        return environment

    async def reason(self, observations: Dict[str, Any]) -> List[Action]:
        """Rule-based reasoning"""
        applicable_actions = []

        for condition, action in self.rules.items():
            if self.evaluate_condition(condition, observations):
                applicable_actions.append(action)

        return applicable_actions

    def evaluate_condition(self, condition: str, observations: Dict[str, Any]) -> bool:
        """Evaluate if a condition is met given current observations"""
        # Simplified condition evaluation
        # In practice, this would be more sophisticated
        try:
            return eval(condition, {"obs": observations, "state": self.state})
        except:
            return False

    async def act(self, action: Action) -> Dict[str, Any]:
        """Execute action directly"""
        print(f"Agent {self.agent_id} executing: {action.name}")
        return {"success": True, "action": action.name}

# Example usage
rules = {
    "obs['temperature'] > 25": Action("turn_on_ac", {}, "Cool the room", 0.9),
    "obs['light_level'] < 0.3": Action("turn_on_lights", {}, "Illuminate room", 0.8),
    "obs['motion_detected'] == True": Action("send_alert", {}, "Alert security", 1.0)
}

reactive_agent = ReactiveAgent("home_automation",
                                AgentState(goals=["maintain_comfort"], beliefs={}),
                                rules)

```

3. Deliberative Agents:

python

```

import heapq
from typing import Tuple

class DeliberativeAgent(Agent):
    """Agent that plans and reasons about future actions"""

    def __init__(self, agent_id: str, initial_state: AgentState, knowledge_base: Dict[str, Any]):
        super().__init__(agent_id, initial_state)
        self.knowledge_base = knowledge_base
        self.planner = ActionPlanner()

    async def perceive(self, environment: Dict[str, Any]) -> Dict[str, Any]:
        """Enhanced perception with filtering and interpretation"""
        relevant_observations = {}

        for key, value in environment.items():
            if self.is_relevant(key, value):
                relevant_observations[key] = self.interpret_observation(key, value)

        return relevant_observations

    def is_relevant(self, key: str, value: Any) -> bool:
        """Determine if an observation is relevant to agent goals"""
        # Check against current goals and knowledge base
        return any(goal_keyword in key.lower() for goal in self.state.goals
                    for goal_keyword in goal.split())

    def interpret_observation(self, key: str, value: Any) -> Any:
        """Interpret raw observation using knowledge base"""
        if key in self.knowledge_base:
            interpretation_rule = self.knowledge_base[key]
            # Apply interpretation rule
            return interpretation_rule.get(value, value)
        return value

    async def reason(self, observations: Dict[str, Any]) -> List[Action]:
        """Plan-based reasoning"""
        # Update world model
        self.update_world_model(observations)

        # Generate plan for current goals
        plan = await self.planner.generate_plan(
            self.state.goals,
            self.state.beliefs,
            self.knowledge_base
        )

```

```

# Convert plan steps to actions
actions = [self.plan_step_to_action(step) for step in plan[:3]] # Next 3 steps
return [action for action in actions if action is not None]

def update_world_model(self, observations: Dict[str, Any]):
    """Update internal model of the world"""
    for key, value in observations.items():
        self.state.beliefs[f"world_{key}"] = value

    # Infer additional facts
    inferred_facts = self.infer_facts(key, value)
    for fact_key, fact_value in inferred_facts.items():
        self.state.beliefs[fact_key] = fact_value

def infer_facts(self, observation_key: str, observation_value: Any) -> Dict[str, Any]:
    """Infer additional facts from observations"""
    inferences = {}

    # Example inference rules
    if observation_key == "weather" and observation_value == "rainy":
        inferences["need_umbrella"] = True
        inferences["traffic_slower"] = True
    elif observation_key == "time" and 18 <= int(observation_value) <= 20:
        inferences["dinner_time"] = True

    return inferences

def plan_step_to_action(self, plan_step: str) -> Optional[Action]:
    """Convert a plan step to an executable action"""
    # Parse plan step and create action
    # This would be more sophisticated in practice
    if "search" in plan_step.lower():
        return Action("search_web", {"query": plan_step, "Find information", 0.7)
    elif "send" in plan_step.lower():
        return Action("send_message", {"content": plan_step, "Communicate", 0.8)
    return None

async def act(self, action: Action) -> Dict[str, Any]:
    """Execute action with planning consideration"""
    # Check preconditions
    if not self.check_preconditions(action):
        return {"success": False, "reason": "Preconditions not met"}

    # Execute action
    result = await self.execute_action(action)

```

```

# Update plan based on result
await self.planner.update_plan_with_result(action, result)

return result

def check_preconditions(self, action: Action) -> bool:
    """Check if action preconditions are satisfied"""
    # Simplified precondition checking
    return True

async def execute_action(self, action: Action) -> Dict[str, Any]:
    """Execute the actual action"""
    print(f"Agent {self.agent_id} executing planned action: {action.name}")
    return {"success": True, "action": action.name, "parameters": action.parameters}

class ActionPlanner:
    """Plans sequences of actions to achieve goals"""

    def __init__(self):
        self.current_plan = []

    async def generate_plan(self, goals: List[str], beliefs: Dict[str, Any],
                           knowledge_base: Dict[str, Any]) -> List[str]:
        """Generate action plan using A* search or similar"""
        plan = []

        for goal in goals:
            goal_plan = self.plan_for_goal(goal, beliefs, knowledge_base)
            plan.extend(goal_plan)

        self.current_plan = plan
        return plan

    def plan_for_goal(self, goal: str, beliefs: Dict[str, Any],
                      knowledge_base: Dict[str, Any]) -> List[str]:
        """Plan actions for a specific goal"""
        # Simplified planning - in practice would use proper search algorithms
        if "find information" in goal.lower():
            return ["search for relevant information", "analyze results", "summarize findings"]
        elif "communicate" in goal.lower():
            return ["prepare message", "send message", "await response"]
        else:
            return [f"work towards {goal}"]

    async def update_plan_with_result(self, action: Action, result: Dict[str, Any]):
        """Update plan based on action results"""
        if not result.get("success", False):

```

```
# Replan if action failed
```

```
print("Action failed, replanning...")
```

```
# In practice, would trigger replanning
```

4. BDI (Belief-Desire-Intention) Agents:

```
python
```

```
@dataclass
```

```
class Belief:
```

```
    content: str
```

```
    confidence: float
```

```
    timestamp: float
```

```
@dataclass
```

```
class Desire:
```

```
    description: str
```

```
    priority: float
```

```
    conditions: List[str]
```

```
@dataclass
```

```
class Intention:
```

```
    goal: str
```

```
    plan: List[Action]
```

```
    commitment_level: float
```

```
    status: str = "active" # active, suspended, completed, abandoned
```

```
class BDIAgent(Agent):
```

```
    """Agent based on Belief-Desire-Intention architecture"""
```

```
    def __init__(self, agent_id: str, initial_beliefs: List[Belief],  
                 initial_desires: List[Desire]):
```

```
        super().__init__(agent_id, AgentState(goals=[], beliefs={}))
```

```
        self.beliefs = {b.content: b for b in initial_beliefs}
```

```
        self.desires = initial_desires
```

```
        self.intentions = []
```

```
        self.intention_stack = []
```

```
    async def perceive(self, environment: Dict[str, Any]) -> Dict[str, Any]:
```

```
        """Perceive and update beliefs"""
```

```
        new_observations = {}
```

```
        for key, value in environment.items():
```

```
            # Create or update belief
```

```
            belief_content = f"{key}={value}"
```

```
            confidence = self.calculate_belief_confidence(key, value)
```

```
            self.beliefs[belief_content] = Belief(
```

```
                content=belief_content,
```

```
                confidence=confidence,
```

```
                timestamp=asyncio.get_event_loop().time()
```

```
            )
```

```
            new_observations[key] = value
```

```

# Prune old or low-confidence beliefs
self.prune_beliefs()

return new_observations

def calculate_belief_confidence(self, key: str, value: Any) -> float:
    """Calculate confidence in a belief based on source and consistency"""
    # Simplified confidence calculation
    base_confidence = 0.8

    # Check consistency with existing beliefs
    consistent_beliefs = sum(1 for b in self.beliefs.values()
                             if self.beliefs_consistent(f"{key}={value}", b.content))

    # Higher consistency increases confidence
    consistency_bonus = min(0.2, consistent_beliefs * 0.05)

    return min(1.0, base_confidence + consistency_bonus)

def beliefs_consistent(self, belief1: str, belief2: str) -> bool:
    """Check if two beliefs are consistent"""
    # Simplified consistency check
    return not (belief1.split('=')[0] == belief2.split('=')[0] and
                belief1.split('=')[1] != belief2.split('=')[1])

def prune_beliefs(self, min_confidence: float = 0.3, max_age: float = 3600):
    """Remove old or low-confidence beliefs"""
    current_time = asyncio.get_event_loop().time()

    to_remove = [content for content, belief in self.beliefs.items()
                  if (belief.confidence < min_confidence or
                      current_time - belief.timestamp > max_age)]

    for content in to_remove:
        del self.beliefs[content]

async def reason(self, observations: Dict[str, Any]) -> List[Action]:
    """BDI reasoning cycle"""
    # 1. Update desires based on current beliefs
    self.update_desires()

    # 2. Select intentions from desires
    self.select_intentions()

    # 3. Plan for active intentions
    actions = []

```

```

for intention in self.intentions:
    if intention.status == "active":
        next_actions = self.get_next_actions(intention)
        actions.extend(next_actions)

return actions

def update_desires(self):
    """Update desires based on current beliefs"""
    for desire in self:

```

AI/ML Interview Questions & Answers Guide

Machine Learning (ML)

1. Explain the bias-variance tradeoff and how it relates to overfitting and underfitting.

Answer:

The bias-variance tradeoff is a fundamental concept in machine learning that describes the relationship between model

- Bias: Error from overly simplistic assumptions in the learning algorithm. High bias leads to underfitting.
- Variance: Error from sensitivity to small fluctuations in the training set. High variance leads to overfitting.
- Total Error = Bias² + Variance + Irreducible Error

Relationship to overfitting/underfitting:

- Underfitting (High Bias, Low Variance): Model is too simple, fails to capture underlying patterns
- Overfitting (Low Bias, High Variance): Model is too complex, memorizes training data but fails on new data
- Sweet Spot: Balance both to minimize total error

Explanation: As model complexity increases, bias decreases but variance increases. The goal is finding the optimal c

2. What are the key differences between Random Forest and Gradient Boosting?

Answer:

Aspect	Random Forest	Gradient Boosting
Training	Parallel (bagging)	Sequential (boosting)
Tree Building	Full depth, random features	Shallow trees, learns from errors
Bias-Variance	Reduces variance	Reduces bias
Overfitting Risk	Lower (averaging effect)	Higher (sequential learning)
Speed	Faster training/prediction	Slower due to sequential nature
Feature Importance	Less reliable	More reliable

Explanation: Random Forest builds multiple independent trees and averages their predictions, reducing variance. G

3. Explain cross-validation and its variants. When would you use each?

Answer:

Cross-Validation Purpose: Estimate model performance on unseen data and select hyperparameters.

Variants:

1. **K-Fold CV:**

- Split data into k equal folds
- Train on k-1 folds, validate on 1
- Repeat k times, average results
- **Use when:** Standard case with sufficient data

2. **Stratified K-Fold:**

- Maintains class distribution in each fold
- **Use when:** Imbalanced datasets

3. **Leave-One-Out (LOO):**

- $k = n$ (number of samples)
- **Use when:** Very small datasets

4. **Time Series Split:**

- Respects temporal order
- **Use when:** Time-dependent data

5. **Group K-Fold:**

- Ensures groups don't appear in both train/validation
- **Use when:** Data has natural groupings (e.g., patients, customers)

Explanation: Choice depends on data characteristics, size, and domain constraints. Always consider the data's inherent structure.

4. How do you handle class imbalance in machine learning?

Answer:

Data-Level Approaches:

1. **Oversampling:** SMOTE, ADASYN
2. **Undersampling:** Random, Tomek links, EditedNearestNeighbours
3. **Combined:** SMOTEENN, SMOTETomek

Algorithm-Level Approaches:

1. **Cost-sensitive learning:** Assign higher costs to minority class errors
2. **Threshold tuning:** Adjust classification threshold
3. **Ensemble methods:** EasyEnsemble, BalanceCascade

Evaluation Adjustments:

- Use precision, recall, F1-score instead of accuracy
- ROC-AUC, PR-AUC curves
- Confusion matrix analysis

****Explanation:**** The choice depends on dataset size, computational resources, and whether you can afford to lose major features.

5. Explain regularization techniques and when to use each.

****Answer:****

****L1 Regularization (Lasso):****

- Penalty: $\lambda \sum |w_i|$
- ****Effect:**** Feature selection (drives coefficients to exactly 0)
- ****Use when:**** Want sparse models, feature selection

****L2 Regularization (Ridge):****

- Penalty: $\lambda \sum w_i^2$
- ****Effect:**** Shrinks coefficients toward 0 but doesn't eliminate
- ****Use when:**** All features are potentially relevant, multicollinearity

****Elastic Net:****

- Combines L1 and L2: $\alpha \sum |w_i| + (1-\alpha) \sum w_i^2$
- ****Use when:**** High-dimensional data with correlated features

****Dropout (Neural Networks):****

- Randomly sets neurons to 0 during training
- ****Use when:**** Deep neural networks, prevents co-adaptation

****Early Stopping:****

- Stop training when validation error increases
- ****Use when:**** Any iterative algorithm

****Explanation:**** Regularization prevents overfitting by adding constraints. L1 for feature selection, L2 for general shrinkage.

Deep Learning (DL)

6. Explain the vanishing gradient problem and how modern architectures address it.

****Answer:****

****Vanishing Gradient Problem:****

Gradients become exponentially small in early layers during backpropagation, making learning extremely slow or impossible.

****Mathematical Cause:****

- Chain rule: $\partial L / \partial w_1 = (\partial L / \partial a_n) \times (\partial a_n / \partial a_{n-1}) \times \dots \times (\partial a_2 / \partial w_1)$
- If partial derivatives < 1 , product approaches 0
- Common with sigmoid/tanh activations and deep networks

****Solutions:****

1. **Residual Connections (ResNet):**

- Skip connections: $H(x) = F(x) + x$
- Gradient flows directly through shortcuts

2. **LSTM/GRU:**

- Gating mechanisms control information flow
- Separate forget/input gates prevent vanishing

3. **Better Activations:**

- ReLU: Gradient = 1 for positive inputs
- Leaky ReLU, ELU, Swish

4. **Normalization:**

- BatchNorm, LayerNorm maintain stable gradients
- Normalize inputs to each layer

5. **Proper Initialization:**

- Xavier/Glorot, He initialization
- Maintain variance across layers

Explanation: Modern deep learning success largely stems from architectural innovations that enable gradient flow in

7. Compare different optimization algorithms (SGD, Adam, RMSprop, etc.).

Answer:

Stochastic Gradient Descent (SGD):

- Update: $w = w - \eta \nabla w$
- **Pros:** Simple, well-understood, good final convergence
- **Cons:** Slow, sensitive to learning rate, poor with sparse data

Momentum:

- Update: $v = \beta v + \eta \nabla w$; $w = w - v$
- **Pros:** Accelerates convergence, reduces oscillations
- **Cons:** Still sensitive to learning rate

RMSprop:

- Adapts learning rate per parameter
- Update: $v = \beta v + (1-\beta)\nabla w^2$; $w = w - \eta \nabla w / \sqrt{v + \epsilon}$
- **Pros:** Good for non-stationary problems, handles sparse gradients
- **Cons:** Can be aggressive in later stages

Adam (Adaptive Moment Estimation):

- Combines momentum and RMSprop
- Maintains both first and second moment estimates
- **Pros:** Generally works well, less hyperparameter tuning
- **Cons:** Can converge to suboptimal solutions, needs learning rate decay

AdamW:

- Adam **with** decoupled weight decay
- **Pros:** Better generalization than Adam
- **Use when:** Training transformers, modern architectures

Comparison Table:

Algorithm	Convergence Speed	Final Performance	Hyperparameter Sensitivity	Best Use Case
SGD	Slow	Excellent	High	Fine-tuning, final stages
Adam	Fast	Good	Low	General purpose, prototyping
AdamW	Fast	Excellent	Low	Transformers, modern architectures

Explanation: Adam **is** often the starting choice, but SGD **with** proper tuning often achieves better final performance.

8. Explain attention mechanisms and their evolution to transformers.

Answer:

Basic Attention Mechanism:

- **Problem:** RNNs compress entire sequence into fixed-size vector (bottleneck)
- **Solution:** Let decoder attend to **all** encoder hidden states

Mathematical Formulation:

- Attention Weights:** $\alpha_i = \text{softmax}(\text{score}(h_i, s))$
- Context Vector:** $c = \sum \alpha_i h_i$
- Output:** $y = f(c, s)$

Evolution:

1. Additive Attention (Bahdanau):

- $\text{score}(h_i, s) = v^T \tanh(W h_i + U s)$
- Learnable alignment function

2. Multiplicative Attention (Luong):

- $\text{score}(h_i, s) = h_i^T s$ (dot product)
- Faster computation

3. Self-Attention:

- Attention within same sequence
- Each position attends to **all** positions

4. Multi-Head Attention:

- Multiple attention heads capture different relationships
- $\text{Attention}(Q, K, V) = \text{softmax}(QK^T / \sqrt{d_k})V$

****5. Transformers:****

- ****Key Innovation:**** Removed recurrence entirely
- ****Components:**** Multi-head self-attention + position encoding + feedforward
- ****Benefits:**** Parallelizable, captures long-range dependencies

****Explanation:**** Attention evolved from solving RNN limitations to becoming the foundation of modern NLP through t

9. What are the different types of neural network layers and their purposes?

****Answer:****

****Core Layers:****

****1. Dense/Fully Connected:****

- ****Purpose:**** Learn complex non-linear mappings
- ****Use:**** Classification heads, feature transformation
- ****Parameters:**** Input_size × Output_size + bias

****2. Convolutional:****

- ****Purpose:**** Learn spatial patterns, translation invariant
- ****Use:**** Image processing, local pattern detection
- ****Key concepts:**** Filters, stride, padding, receptive field

****3. Recurrent (RNN/LSTM/GRU):****

- ****Purpose:**** Process sequential data, maintain memory
- ****Use:**** Time series, text processing
- ****Challenge:**** Vanishing gradients (solved by LSTM/GRU)

****4. Attention/Transformer:****

- ****Purpose:**** Capture long-range dependencies
- ****Use:**** NLP, sequence-to-sequence tasks
- ****Advantage:**** Parallelizable, better long-term memory

****Regularization Layers:****

****5. Dropout:****

- ****Purpose:**** Prevent overfitting
- ****Mechanism:**** Randomly set neurons to 0 during training

****6. Batch Normalization:****

- ****Purpose:**** Stabilize training, enable higher learning rates
- ****Mechanism:**** Normalize inputs to each layer

****Pooling Layers:****

****7. MaxPool/AvgPool:****

- ****Purpose:**** Reduce spatial dimensions, add translation invariance

- **Use:** CNNs for dimension reduction

Activation Layers:

8. ReLU/Sigmoid/Tanh:

- **Purpose:** Introduce non-linearity
- **Choice:** ReLU for hidden layers, sigmoid/softmax for output

Explanation: Layer choice depends on data type, task, and architectural requirements. Modern networks combine m

10. How do you choose the right loss function for different tasks?

Answer:

Classification Tasks:

Binary Classification:

- **Binary Cross-Entropy:** Standard choice
- **Focal Loss:** For imbalanced datasets
- **Hinge Loss:** For SVMs, maximum margin

Multi-class Classification:

- **Categorical Cross-Entropy:** Standard for mutually exclusive classes
- **Sparse Categorical Cross-Entropy:** When labels are integers
- **Label Smoothing:** Regularization technique

Multi-label Classification:

- **Binary Cross-Entropy:** Applied to each class independently
- **Hamming Loss:** Fraction of wrong labels

Regression Tasks:

Mean Squared Error (MSE):

- **Use:** Standard regression, sensitive to outliers
- **When:** Gaussian noise assumption

Mean Absolute Error (MAE):

- **Use:** Robust to outliers
- **When:** Laplacian noise, median prediction desired

Huber Loss:

- **Use:** Combines MSE and MAE benefits
- **When:** Some outliers expected

Special Cases:

Contrastive Loss:

- **Use:** Similarity learning, face verification
- **Purpose:** Learn embeddings where similar items are close

Triplet Loss:

- **Use:** Metric learning
- **Purpose:** Anchor-positive closer than anchor-negative

Wasserstein Loss:

- **Use:** GANs
- **Purpose:** Better convergence properties

Selection Criteria:

1. **Task type** (classification/regression)
2. **Data characteristics** (outliers, imbalance)
3. **Business requirements** (precision vs recall)
4. **Model architecture** compatibility

Explanation: Loss function choice significantly impacts model behavior and training dynamics. Always align with task

Natural Language Processing (NLP)

11. Explain the evolution from Word2Vec to BERT to GPT.

Answer:

Word2Vec (2013):

- **Innovation:** Dense vector representations of words
- **Types:** Skip-gram, CBOW
- **Limitation:** Static embeddings, no context awareness
- **Example:** "bank" has same embedding in "river bank" and "savings bank"

Evolution Path:

1. Contextual Embeddings:

- **ELMo (2018):** BiLSTM-based, different representations per context
- **Problem:** Still limited by LSTM constraints

2. BERT (2018):

- **Innovation:** Bidirectional transformer, pre-training + fine-tuning
- **Architecture:** Encoder-only transformer
- **Training:** Masked Language Model + Next Sentence Prediction
- **Strengths:**
 - Bidirectional context
 - Transfer learning paradigm
 - Strong performance on understanding tasks

3. GPT Series:

- **GPT-1 (2018):** Decoder-only transformer, autoregressive
- **GPT-2 (2019):** Scaled up, showed emergent capabilities
- **GPT-3 (2020):** 175B parameters, few-shot learning
- **GPT-4 (2023):** Multimodal, enhanced reasoning

Key Differences:

Aspect	Word2Vec	BERT	GPT
Context	Static	Bidirectional	Unidirectional
Training	Unsupervised	Pre-train + Fine-tune	Autoregressive
Use Case	Word similarity	Understanding	Generation
Architecture	Shallow NN	Encoder	Decoder

Impact:

- **Word2Vec:** Established dense representations
- **BERT:** Dominated understanding tasks via fine-tuning
- **GPT:** Revolutionized generation **and** few-shot learning

Explanation: Each breakthrough addressed limitations of predecessors **while** introducing new capabilities, leading to

12. What are the key components of transformer architecture?

Answer:

Core Components:

1. Multi-Head Self-Attention:

$$\text{Attention}(Q,K,V) = \text{softmax}(QK^T/\sqrt{d_k})V$$

$$\text{MultiHead} = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W^O$$

- **Purpose:** Capture different types of relationships
- **Benefit:** Parallel processing, long-range dependencies

2. Position Encoding:

- **Problem:** Transformers have no inherent position awareness
- **Solution:** Add sinusoidal or learned position embeddings
- **Formula:** $PE(pos, 2i) = \sin(pos/10000^{(2i/d_{model})})$

3. Feed-Forward Networks:

- **Structure:** Two linear layers with ReLU: $FFN(x) = \max(0, xW_1 + b_1)W_2 + b_2$
- **Purpose:** Add non-linearity and model complexity
- **Size:** Usually 4× hidden dimension

4. Layer Normalization:

- **Placement:** Pre-norm (before attention/FFN) or post-norm
- **Purpose:** Stabilize training, enable deeper networks
- **Formula:** $LayerNorm(x) = \gamma(x - \mu)/\sigma + \beta$

5. Residual Connections:

- **Structure:** $output = LayerNorm(x + Sublayer(x))$
- **Purpose:** Enable gradient flow, deeper networks

Architecture Variants:

Encoder-Only (BERT):

- Bidirectional attention
- Good for understanding tasks
- Masked language modeling

Decoder-Only (GPT):

- Causal (masked) attention
- Good for generation
- Autoregressive training

Encoder-Decoder (T5):

- Full attention in encoder
- Causal attention in decoder
- Good for seq2seq tasks

Key Innovations:

1. **Parallelization:** No sequential dependencies
2. **Attention:** Direct modeling of all pairwise relationships
3. **Scalability:** Efficient training on large datasets
4. **Transfer Learning:** Pre-train once, fine-tune for many tasks

Explanation: Transformers' success stems from combining attention mechanisms with architectural innovations

that enable efficient training and strong performance across diverse tasks.

13. How do you handle different NLP tasks (classification, NER, QA, summarization)?

Answer:

Text Classification:

Approach:

- Add classification head to pre-trained model
- Fine-tune entire model or freeze backbone

Architecture:

Input → BERT/RoBERTa → [CLS] token → Dense → Softmax

Considerations:

- Multi-label vs multi-class
- Imbalanced datasets (focal loss, class weights)
- Hierarchical classification for large taxonomies

Named Entity Recognition (NER):

Approach:

- Token-level classification
- BIOES-style tagging schemes

Architecture:

Input → BERT → Token representations → Dense → CRF (optional)

****Challenges:****

- Nested entities
- Domain adaptation
- Few-shot learning for new entity types

****Question Answering:****

****Types:****

1. ****Extractive QA:**** Select span from context
2. ****Abstractive QA:**** Generate answer
3. ****Open-domain QA:**** Retrieve + generate

****Extractive Approach:****

Question + Context → BERT → Start/End position logits

****Abstractive Approach:****

Question + Context → T5/BART → Generated answer

****Text Summarization:****

****Types:****

1. ****Extractive:**** Select important sentences
2. ****Abstractive:**** Generate new summary

****Extractive Methods:****

- Sentence scoring (TF-IDF, TextRank)
- Classification approach (sentence-level)

****Abstractive Methods:****

- Encoder-decoder models (T5, BART, Pegasus)
- Pointer-generator networks
- Copy mechanisms for handling OOV words

****Evaluation Metrics:****

- ****ROUGE:**** Overlap-based (summarization, QA)
- ****BLEU:**** N-gram precision (generation)
- ****BERTScore:**** Semantic similarity
- ****F1:**** Token-level (NER, extractive QA)

****Common Strategies:****

1. ****Transfer Learning:**** Start with pre-trained models
2. ****Data Augmentation:**** Paraphrasing, back-translation
3. ****Multi-task Learning:**** Joint training on related tasks
4. ****Active Learning:**** Efficient annotation strategies

****Explanation:**** Modern NLP leverages pre-trained transformers as foundation, with task-specific heads and training strategies. The key is adapting the architecture and training procedure to task requirements.

14. What are the challenges in multilingual NLP and how do you address them?

****Answer:****

****Key Challenges:****

****1. Script and Writing System Diversity:****

- Different alphabets (Latin, Cyrillic, Arabic, Chinese)
- Right-to-left vs left-to-right
- ****Solution:**** Unicode normalization, script-aware preprocessing

****2. Morphological Complexity:****

- Agglutinative languages (Turkish, Finnish)
- Rich inflection systems
- ****Solution:**** Subword tokenization (BPE, SentencePiece), morphological analyzers

****3. Data Imbalance:****

- High-resource vs low-resource languages
- English dominance in training data
- ****Solution:**** Transfer learning, multilingual pre-training, data augmentation

****4. Code-switching:****

- Mixed languages within text
- ****Solution:**** Language identification, specialized models

****Multilingual Approaches:****

****1. Multilingual Pre-trained Models:****

- ****mBERT:**** BERT trained on 104 languages
- ****XLM-R:**** RoBERTa-based, better performance
- ****mT5:**** Multilingual T5 for generation tasks

****2. Cross-lingual Transfer:****

- ****Zero-shot:**** Train on source language, test on target
- ****Few-shot:**** Limited target language data
- ****Translate-train:**** Translate training data to target language
- ****Translate-test:**** Translate test data to source language

****3. Language-Specific Adaptations:****

- ****Continued pre-training:**** Adapt multilingual model to target language
- ****Language adapters:**** Lightweight adaptation modules
- ****Language-specific tokenizers:**** Better subword segmentation

****Technical Solutions:****

****Tokenization:****

- ****SentencePiece:**** Language-agnostic subword tokenization
- ****Shared vocabulary:**** Balance between languages
- ****Language-specific tokens:**** [LANG] prefixes

****Alignment Techniques:****

- ****Word alignment:**** Statistical or neural alignment models
- ****Sentence alignment:**** For parallel corpora
- ****Cross-lingual embeddings:**** Map embeddings across languages

****Evaluation Considerations:****

- ****XNLI, XTREME:**** Cross-lingual benchmarks
- ****Language-specific metrics:**** Consider linguistic phenomena
- ****Cultural bias:**** Ensure fair evaluation across cultures

****Best Practices:****

1. ****Start with multilingual pre-trained models****

2. **Use language identification for mixed text**
3. **Consider script normalization**
4. **Validate on native speaker annotations**
5. **Address cultural and linguistic biases**

Explanation: Multilingual NLP requires understanding linguistic diversity and leveraging transfer learning techniques while being mindful of cultural and resource imbalances across languages.

15. Explain different text preprocessing techniques and when to use them.

Answer:

Basic Preprocessing:

1. Text Cleaning:

- **Remove HTML tags:** `<p>`, `<div>`, etc.
- **Handle URLs:** Remove or replace with tokens
- **Email addresses:** Replace with [EMAIL] token
- **When to use:** Web scraping, social media data

2. Normalization:

- **Lowercasing:** Standard for many tasks
- **Unicode normalization:** Handle different encodings
- **Accent removal:** Convert é → e
- **When NOT to use:** Named entities, case-sensitive tasks

3. Whitespace and Punctuation:

- **Strip extra whitespace:** Multiple spaces → single space
- **Punctuation handling:**
 - Remove for topic modeling
 - Keep for sentiment (!!!, ???)
 - Separate from words (tokenization)

Linguistic Preprocessing:

4. Tokenization:

- **Word-level:** Split by spaces/punctuation
- **Subword:** BPE, WordPiece, SentencePiece
- **Sentence-level:** Split documents into sentences
- **Choice depends on:** Model architecture, language characteristics

5. Stop Word Removal:

- **Remove:** "the", "and", "is", etc.
- **When to use:** Topic modeling, keyword extraction
- **When NOT to use:** Sentiment analysis, machine translation

6. Stemming vs Lemmatization:

****Stemming (Porter, Snowball):****

- ****Process:**** Remove suffixes algorithmically
- ****Example:**** running, runs, ran → run
- ****Pros:**** Fast, simple
- ****Cons:**** May not produce real words

****Lemmatization:****

- ****Process:**** Dictionary-based reduction to base form
- ****Example:**** better → good, went → go
- ****Pros:**** Produces real words, more accurate
- ****Cons:**** Slower, requires linguistic resources

****Advanced Preprocessing:****

****7. Part-of-Speech Tagging:****

- ****Purpose:**** Identify grammatical roles
- ****Use cases:**** Information extraction, grammar correction
- ****Tools:**** spaCy, NLTK, Stanza

****8. Named Entity Recognition:****

- ****Purpose:**** Identify person, location, organization names
- ****Use cases:**** Information extraction, anonymization
- ****Approach:**** Rule-based or ML-based

****9. Spell Correction:****

- ****Methods:**** Edit distance, statistical, neural
- ****When to use:**** Noisy text (social media, OCR)
- ****Trade-offs:**** May change meaning, computational cost

****Modern Approaches:****

****10. Subword Tokenization:****

- ****BPE (Byte Pair Encoding):**** Merge frequent character pairs
- ****WordPiece:**** Used by BERT, probability-based merging
- ****SentencePiece:**** Language-agnostic, handles raw text

****Task-Specific Considerations:****

****Text Classification:****

- Minimal preprocessing with transformers
- Traditional ML: full preprocessing pipeline

****Machine Translation:****

- Preserve punctuation, capitalization
- Language-specific normalizations

****Information Retrieval:****

- Stemming for broader matching
- Keep stop words for phrase queries

****Sentiment Analysis:****

- Preserve emoticons, punctuation (!!!)
- Handle negations carefully

****Decision Framework:****

1. ****Model type:**** Traditional ML needs more preprocessing
2. ****Data quality:**** Noisy data needs more cleaning
3. ****Task requirements:**** Preserve relevant signals
4. ****Language characteristics:**** Morphologically rich languages
5. ****Computational resources:**** Complex preprocessing costs

****Explanation:**** Preprocessing should align with model capabilities and task requirements. Modern transformer models need less preprocessing, but traditional ML approaches benefit from comprehensive text cleaning and normalization.

MLOps

16. Explain the key components of an MLOps pipeline.

****Answer:****

****MLOps Pipeline Components:****

****1. Data Management:****

- ****Data Ingestion:**** Batch/streaming data collection
- ****Data Validation:**** Schema validation, quality checks
- ****Data Versioning:**** Track data changes (DVC, MLflow)
- ****Data Lineage:**** Track data transformations and sources
- ****Feature Store:**** Centralized feature management

****2. Model Development:****

- ****Experiment Tracking:**** MLflow, Weights & Biases, Neptune
- ****Version Control:**** Git for code, specialized tools for models
- ****Reproducible Environments:**** Docker, Conda environments
- ****Hyperparameter Tuning:**** Optuna, Hyperopt, Ray Tune

****3. Model Training Pipeline:****

- ****Training Orchestration:**** Airflow, Kubeflow, Prefect
- ****Resource Management:**** Auto-scaling, GPU allocation
- ****Distributed Training:**** Multi-GPU, multi-node training
- ****Training Monitoring:**** Loss curves, resource utilization

****4. Model Validation & Testing:****

- **Model Evaluation:** Automated testing on validation sets
- **A/B Testing Framework:** Compare model versions
- **Bias Detection:** Fairness metrics, demographic parity
- **Performance Regression Testing:** Ensure new models don't degrade

5. Model Deployment:

- **Serving Infrastructure:** REST APIs, gRPC services
- **Containerization:** Docker, Kubernetes
- **Scaling:** Auto-scaling based on load
- **Multi-model Serving:** TensorFlow Serving, Seldon, KFServing

6. Monitoring & Observability:

- **Model Performance:** Accuracy drift, prediction quality
- **Data Drift:** Input distribution changes
- **System Metrics:** Latency, throughput, resource usage
- **Business Metrics:** Revenue impact, user satisfaction

7. Model Lifecycle Management:

- **Model Registry:** Centralized model catalog
- **Promotion Workflow:** Dev → Staging → Production
- **Rollback Mechanisms:** Quick reversion for issues
- **Model Retirement:** Decommission outdated models

Pipeline Automation:

Data Pipeline → Feature Engineering → Training → Validation →
Deployment → Monitoring → Retraining (if needed)

****Key Principles:****

1. ****Automation:**** Minimize manual interventions
2. ****Reproducibility:**** Ensure consistent results
3. ****Scalability:**** Handle increasing data/model complexity
4. ****Reliability:**** High availability, fault tolerance
5. ****Observability:**** Comprehensive monitoring and alerting

****Tools Ecosystem:****

- ****Orchestration:**** Airflow, Kubeflow Pipelines
- ****Serving:**** TensorFlow Serving, MLflow Models
- ****Monitoring:**** Evidently, WhyLabs, Great Expectations
- ****Infrastructure:**** Kubernetes, Cloud ML platforms

****Explanation:**** MLOps pipelines automate the entire ML lifecycle, ensuring reliable, scalable, and maintainable ML systems in production. The key is integrating all components seamlessly while maintaining observability and reproducibility.

17. How do you handle model versioning and experiment tracking?

****Answer:****

****Model Versioning Strategies:****

****1. Semantic Versioning:****

- ****Format:**** MAJOR.MINOR.PATCH (e.g., 2.1.3)
- ****MAJOR:**** Breaking changes, different architecture
- ****MINOR:**** New features, backward compatible
- ****PATCH:**** Bug fixes, parameter updates

****2. Git-based Versioning:****

- ****Code + Model:**** Store model artifacts with code commits
- ****Git LFS:**** Handle large model files
- ****Tags:**** Mark model releases
- ****Branches:**** Separate model variants

****3. Metadata-based Versioning:****

- ****Training Data Version:**** Dataset hash/version
- ****Hyperparameters:**** Complete configuration
- ****Environment:**** Software versions, hardware specs
- ****Performance Metrics:**** Accuracy, loss values

****Model Storage Solutions:****

****1. Model Registry (MLflow Model Registry):****

```
``python
```

```
# Register model
mlflow.register_model(
    model_uri="runs:/<run_id>/model",
    name="fraud_detection_model",
    tags={"algorithm": "xgboost", "version": "2.1.0"}
)

# Transition to production
client.transition_model_version_stage(
    name="fraud_detection_model",
    version=3,
    stage="Production"
)
```

2. Cloud Storage with Metadata:

- **S3/GCS:** Store model artifacts
- **Database:** Store metadata and lineage
- **API Layer:** Access and management interface

Experiment Tracking:

1. MLflow Tracking:

```
python

import mlflow

with mlflow.start_run():
    # Log parameters
    mlflow.log_param("learning_rate", 0.01)
    mlflow.log_param("max_depth", 6)

    # Train model
    model = train_model(X_train, y_train)

    # Log metrics
    mlflow.log_metric("accuracy", accuracy)
    mlflow.log_metric("f1_score", f1)

    # Log model
    mlflow.sklearn.log_model(model, "model")

    # Log artifacts
    mlflow.log_artifact("confusion_matrix.png")
```

2. Weights & Biases:

```
python

import wandb

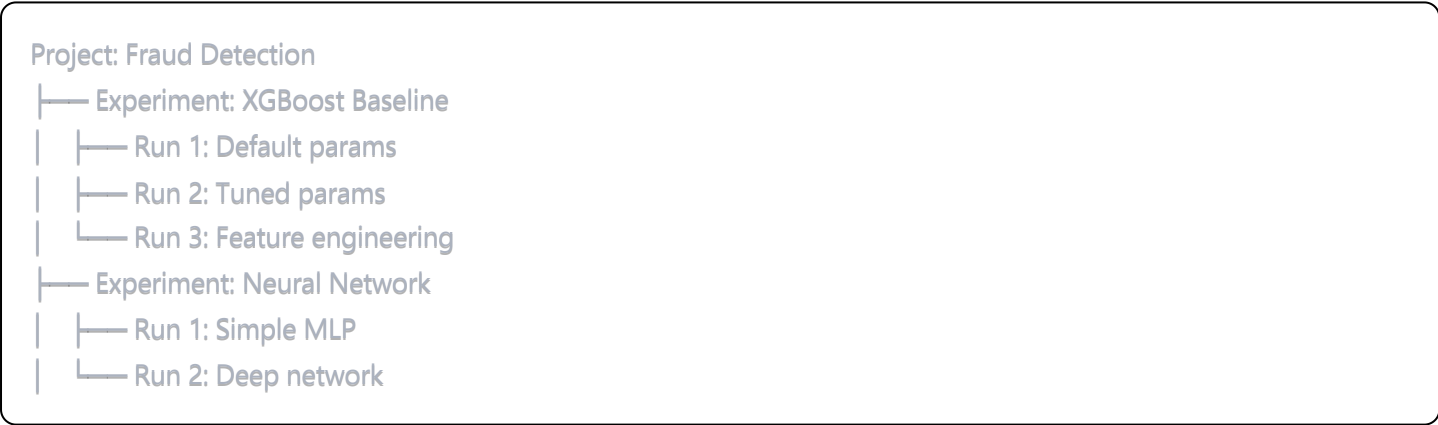
wandb.init(project="my-project")
wandb.config.update({"learning_rate": 0.01, "epochs": 100})

for epoch in range(epochs):
    # Training loop
    train_loss = train_step()
    val_loss = validate()

    wandb.log({
        "epoch": epoch,
        "train_loss": train_loss,
        "val_loss": val_loss
    })
```

Experiment Organization:

1. Hierarchical Structure:



2. Tagging System:

- **Algorithm tags:** "xgboost", "neural_network"
- **Feature tags:** "text_features", "numerical_only"
- **Status tags:** "baseline", "production_candidate"
- **Data tags:** "v1_dataset", "augmented_data"

Best Practices:

1. Reproducibility:

```
python
```

```
# Log everything needed for reproduction
mlflow.log_param("random_seed", 42)
mlflow.log_param("data_version", "v1.2.3")
mlflow.log_param("preprocessing_steps", ["normalize", "pca"])
mlflow.log_artifact("requirements.txt")
mlflow.log_artifact("config.yaml")
```

2. Model Comparison:

- **A/B Testing:** Compare model versions in production
- **Champion/Challenger:** New model vs current best
- **Performance Tracking:** Monitor metrics over time

3. Automated Versioning:

```
python

def auto_version_model(model, metrics, threshold=0.95):
    if metrics['accuracy'] > threshold:
        version = get_next_version()
        register_model(model, version, metrics)
        return version
    else:
        log_failed_experiment(model, metrics)
        return None
```

Model Lineage Tracking:

- **Data Lineage:** Which datasets were used
- **Code Lineage:** Which code version produced the model
- **Parameter Lineage:** How hyperparameters evolved
- **Performance Lineage:** Metric changes over versions

Deployment Integration:

```
python
```

```
# Automated deployment based on validation
```

```
def deploy_if_better(new_model_version, current_version):  
    new_metrics = evaluate_model(new_model_version)  
    current_metrics = get_production_metrics(current_version)  
  
    if new_metrics['f1'] > current_metrics['f1'] * 1.05: # 5% improvement  
        deploy_model(new_model_version, stage="production")  
        demote_model(current_version, stage="archived")
```

Explanation: Effective model versioning and experiment tracking require systematic organization, comprehensive metadata capture, and integration with deployment workflows. This enables reproducibility, collaboration, and continuous improvement in ML systems.

18. What strategies do you use for monitoring ML models in production?

Answer:

Types of ML Monitoring:

1. Data Quality Monitoring:

Schema Validation:

```
python  
  
# Example with Great Expectations  
import great_expectations as ge  
  
# Define expectations  
df = ge.from_pandas(production_data)  
df.expect_column_to_exist("feature_1")  
df.expect_column_values_to_be_between("age", 18, 100)  
df.expect_column_values_to_not_be_null("customer_id")  
  
# Validate and alert  
validation_result = df.validate()  
if not validation_result.success:  
    send_alert("Data quality issues detected")
```

Statistical Drift Detection:

- **Distribution Changes:** Kolmogorov-Smirnov test, Chi-square test
- **Statistical Moments:** Mean, variance, skewness changes
- **Population Stability Index (PSI):** Measure distribution shifts

2. Model Performance Monitoring:

Real-time Prediction Monitoring:

```
python

# Monitor prediction distributions
def monitor_predictions(predictions):
    current_mean = np.mean(predictions)
    current_std = np.std(predictions)

    # Compare with baseline statistics
    if abs(current_mean - baseline_mean) > 2 * baseline_std:
        alert("Prediction distribution drift detected")

    # Monitor prediction confidence
    if hasattr(predictions, 'probability'):
        low_confidence_rate = sum(max(p) < 0.7 for p in predictions.probability)
        if low_confidence_rate > threshold:
            alert("High uncertainty in predictions")
```

Performance Degradation Detection:

```
python

# When ground truth is available (delayed)
def check_model_performance(predictions, actuals, window_size=1000):
    recent_accuracy = accuracy_score(actuals[-window_size:],
                                     predictions[-window_size:])
    baseline_accuracy = get_baseline_accuracy()

    if recent_accuracy < baseline_accuracy * 0.95: # 5% degradation
        trigger_retraining_pipeline()
        alert("Model performance degraded")
```

3. System Performance Monitoring:

Latency and Throughput:

```
python
```

```
import time
from prometheus_client import Histogram, Counter

# Metrics collection
REQUEST_LATENCY = Histogram('ml_request_duration_seconds',
                             'ML request latency')
REQUEST_COUNT = Counter('ml_requests_total', 'Total ML requests')

@REQUEST_LATENCY.time()
def predict(data):
    start_time = time.time()

    # Model inference
    prediction = model.predict(data)

    # Log metrics
    latency = time.time() - start_time
    REQUEST_COUNT.inc()

    if latency > SLA_THRESHOLD:
        alert(f"High latency detected: {latency:.2f}s")

    return prediction
```

Resource Utilization:

- **CPU/GPU Usage:** Monitor inference load
- **Memory Usage:** Track memory leaks, OOM errors
- **Disk I/O:** Model loading, feature store access

4. Business Metrics Monitoring:

Domain-specific KPIs:

python

```
# E-commerce recommendation system
```

```
def monitor_business_metrics():
```

```
    click_through_rate = get_ctr_last_24h()
```

```
    conversion_rate = get_conversion_rate_last_24h()
```

```
    revenue_per_recommendation = get_revenue_metrics()
```

```
    if click_through_rate < baseline_ctr * 0.9:
```

```
        alert("CTR significantly dropped")
```

```
# A/B test monitoring
```

```
control_group_performance = get_control_metrics()
```

```
treatment_group_performance = get_treatment_metrics()
```

```
significance = statistical_test(control_group_performance,  
                                treatment_group_performance)
```

```
if significance < 0.05:
```

```
    log_experiment_result("statistically_significant")
```

Monitoring Infrastructure:

1. Metrics Collection:

- **Prometheus + Grafana:** Time-series metrics and dashboards
- **ELK Stack:** Log aggregation and analysis
- **Custom Dashboards:** Business-specific visualizations

2. Alerting System:

```
python
```

```
# Multi-level alerting
```

```
class AlertManager:
```

```
    def __init__(self):
```

```
        self.thresholds = {
```

```
            'warning': {'accuracy_drop': 0.05, 'latency_increase': 0.2},
```

```
            'critical': {'accuracy_drop': 0.10, 'latency_increase': 0.5}
```

```
        }
```

```
    def evaluate_alert(self, metric_name, current_value, baseline_value):
```

```
        change = abs(current_value - baseline_value) / baseline_value
```

```
        if change > self.thresholds['critical'][metric_name]:
```

```
            self.send_alert('critical', metric_name, change)
```

```
        elif change > self.thresholds['warning'][metric_name]:
```

```
            self.send_alert('warning', metric_name, change)
```

3. Automated Response:

```
python

# Automated remediation
def automated_response(alert_type, metric_name):
    if alert_type == 'critical' and metric_name == 'accuracy_drop':
        # Rollback to previous model version
        rollback_model()

    elif alert_type == 'warning' and metric_name == 'data_drift':
        # Trigger retraining pipeline
        schedule_retraining()

    elif metric_name == 'high_latency':
        # Scale up inference servers
        scale_inference_pods(replicas=current_replicas * 2)
```

Advanced Monitoring Techniques:

1. Drift Detection Methods:

- **Adversarial Validation:** Train classifier to distinguish train/serve data
- **Domain Adaptation:** Measure domain shift
- **Uncertainty Quantification:** Monitor model confidence

2. Explainability Monitoring:

```
python

# Monitor feature importance changes
def monitor_feature_importance():
    current_importance = get_feature_importance(current_model)
    baseline_importance = get_baseline_importance()

    # Detect significant shifts in feature usage
    importance_shift = calculate_shift(current_importance, baseline_importance)
    if importance_shift > threshold:
        alert("Feature importance significantly changed")
```

Best Practices:

1. **Establish Baselines:** Use training/validation performance as reference
2. **Multiple Monitoring Layers:** Data, model, system, business metrics
3. **Gradual Rollouts:** Canary deployments for new models

4. **Feedback Loops:** Use monitoring insights for model improvement

5. **Documentation:** Maintain runbooks for common issues

Explanation: Comprehensive ML monitoring requires tracking multiple dimensions - from data quality to business impact. The key is building automated systems that can detect issues early and trigger appropriate responses, ensuring reliable ML systems in production.

19. How do you implement A/B testing for ML models?

Answer:

A/B Testing Framework for ML Models:

1. Experimental Design:

Traffic Splitting Strategies:

```
python

import hashlib

class TrafficSplitter:
    def __init__(self, experiment_config):
        self.experiments = experiment_config

    def assign_variant(self, user_id, experiment_name):
        # Consistent hash-based assignment
        hash_input = f"{user_id}_{experiment_name}"
        hash_value = int(hashlib.md5(hash_input.encode()).hexdigest(), 16)

        # Determine assignment based on hash
        assignment_ratio = hash_value % 100

        if assignment_ratio < self.experiments[experiment_name]['control_ratio']:
            return 'control'
        else:
            return 'treatment'

# Example usage
splitter = TrafficSplitter({
    'model_v2_test': {'control_ratio': 50} # 50-50 split
})

variant = splitter.assign_variant(user_id="12345", experiment_name="model_v2_test")
```

Stratified Randomization:

python

```
def stratified_assignment(user_features, strata_config):  
    """Ensure balanced assignment across important user segments"""  
  
    # Define strata (e.g., by user segment, geography)  
    if user_features['country'] == 'US' and user_features['tier'] == 'premium':  
        stratum = 'us_premium'  
    elif user_features['country'] == 'US' and user_features['tier'] == 'basic':  
        stratum = 'us_basic'  
    # ... other strata  
  
    # Assign within stratum  
    return assign_within_stratum(user_features['user_id'], stratum)
```

2. Infrastructure Implementation:

Feature Flag Integration:

python

```
from launchdarkly import LDClient
```

```
class ModelSelector:
```

```
    def __init__(self):
```

```
        self.ld_client = LDClient("your-sdk-key")
```

```
        self.models = {
```

```
            'control': load_model('model_v1.pkl'),
```

```
            'treatment': load_model('model_v2.pkl')
```

```
        }
```

```
    def get_prediction(self, user_context, features):
```

```
        # Get experiment assignment
```

```
        variant = self.ld_client.variation(
```

```
            'model_ab_test',
```

```
            user_context,
```

```
            'control' # default
```

```
        )
```

```
        # Select model based on assignment
```

```
        model = self.models[variant]
```

```
        prediction = model.predict(features)
```

```
        # Log for analysis
```

```
        self.log_prediction(user_context['key'], variant, prediction, features)
```

```
    return prediction
```

Canary Deployment Pattern:

```
python
```

```
class CanaryDeployment:
    def __init__(self, canary_percentage=5):
        self.canary_percentage = canary_percentage
        self.control_model = load_model('production_model')
        self.canary_model = load_model('candidate_model')

    def predict(self, user_id, features):
        if self.is_canary_user(user_id):
            prediction = self.canary_model.predict(features)
            variant = 'canary'
        else:
            prediction = self.control_model.predict(features)
            variant = 'control'

        # Track metrics for both models
        self.track_prediction(user_id, variant, prediction, features)
        return prediction

    def is_canary_user(self, user_id):
        return hash(user_id) % 100 < self.canary_percentage
```

3. Metrics Collection and Analysis:

Metric Tracking System:

```
python
```

```

import time
from dataclasses import dataclass
from typing import Dict, Any

@dataclass
class ExperimentEvent:
    user_id: str
    variant: str
    timestamp: float
    event_type: str # 'prediction', 'click', 'conversion'
    event_data: Dict[str, Any]

class ExperimentTracker:
    def __init__(self):
        self.events = []

    def track_prediction(self, user_id, variant, prediction, features, model_confidence=None):
        event = ExperimentEvent(
            user_id=user_id,
            variant=variant,
            timestamp=time.time(),
            event_type='prediction',
            event_data={
                'prediction': prediction,
                'features': features,
                'model_confidence': model_confidence
            }
        )
        self.events.append(event)
        self.send_to_analytics_system(event)

    def track_outcome(self, user_id, variant, outcome_value):
        """Track business outcome (conversion, revenue, etc.)"""
        event = ExperimentEvent(
            user_id=user_id,
            variant=variant,
            timestamp=time.time(),
            event_type='outcome',
            event_data={'value': outcome_value}
        )
        self.events.append(event)

```

Statistical Analysis:

python

```

import scipy.stats as stats
import numpy as np

class ExperimentAnalyzer:
    def __init__(self, significance_level=0.05):
        self.alpha = significance_level

    def analyze_continuous_metric(self, control_values, treatment_values):
        """Analyze metrics like revenue, latency, etc."""

        # Two-sample t-test
        t_stat, p_value = stats.ttest_ind(treatment_values, control_values)

        # Effect size (Cohen's d)
        pooled_std = np.sqrt(
            ((len(control_values) - 1) * np.var(control_values, ddof=1) +
             (len(treatment_values) - 1) * np.var(treatment_values, ddof=1)) /
            (len(control_values) + len(treatment_values) - 2)
        )
        cohens_d = (np.mean(treatment_values) - np.mean(control_values)) / pooled_std

        # Confidence interval
        se = pooled_std * np.sqrt(1/len(control_values) + 1/len(treatment_values))
        ci_lower, ci_upper = stats.t.interval(
            1 - self.alpha,
            len(control_values) + len(treatment_values) - 2,
            loc=np.mean(treatment_values) - np.mean(control_values),
            scale=se
        )

        return {
            'p_value': p_value,
            'significant': p_value < self.alpha,
            'effect_size': cohens_d,
            'confidence_interval': (ci_lower, ci_upper),
            'treatment_mean': np.mean(treatment_values),
            'control_mean': np.mean(control_values)
        }

    def analyze_binary_metric(self, control_conversions, control_total,
                             treatment_conversions, treatment_total):
        """Analyze binary metrics like click-through rate"""

        control_rate = control_conversions / control_total
        treatment_rate = treatment_conversions / treatment_total

```

```
# Chi-square test
contingency_table = np.array([
    [control_conversions, control_total - control_conversions],
    [treatment_conversions, treatment_total - treatment_conversions]
])

chi2, p_value, dof, expected = stats.chi2_contingency(contingency_table)

# Relative lift
relative_lift = (treatment_rate - control_rate) / control_rate

return {
    'p_value': p_value,
    'significant': p_value < self.alpha,
    'control_rate': control_rate,
    'treatment_rate': treatment_rate,
    'relative_lift': relative_lift
}
```

4. Experiment Lifecycle Management:

Automated Experiment Management:

python

```

class ExperimentManager:
    def __init__(self):
        self.active_experiments = {}
        self.analyzer = ExperimentAnalyzer()

    def create_experiment(self, experiment_config):
        """Start new A/B test"""
        experiment_id = experiment_config['id']
        self.active_experiments[experiment_id] = {
            'config': experiment_config,
            'start_time': time.time(),
            'status': 'running',
            'sample_size': 0
        }

    def check_experiment_status(self, experiment_id):
        """Check if experiment should be stopped"""
        experiment = self.active_experiments[experiment_id]

        # Check minimum sample size
        if experiment['sample_size'] < experiment['config']['min_sample_size']:
            return 'continue'

        # Check maximum duration
        if time.time() - experiment['start_time'] > experiment['config']['max_duration']:
            return 'stop_timeout'

        # Check for statistical significance
        results = self.get_experiment_results(experiment_id)
        if results['primary_metric']['significant']:
            return 'stop_significant'

        return 'continue'

    def early_stopping_check(self, experiment_id):
        """Sequential testing for early stopping"""
        # Implement sequential probability ratio test (SPRT)
        # or Bayesian approaches for early stopping
        pass

```

5. Advanced A/B Testing Techniques:

Multi-Armed Bandit:

python

```

import numpy as np

class ThompsonSampling:
    """Adaptive allocation based on performance"""

    def __init__(self, n_arms=2):
        self.n_arms = n_arms
        self.alpha = np.ones(n_arms) # Success counts + 1
        self.beta = np.ones(n_arms) # Failure counts + 1

    def select_arm(self):
        # Sample from Beta distributions
        samples = np.random.beta(self.alpha, self.beta)
        return np.argmax(samples)

    def update(self, arm, reward):
        if reward:
            self.alpha[arm] += 1
        else:
            self.beta[arm] += 1

```

Multivariate Testing:

```

python

def factorial_design_assignment(user_id, factors):
    """Assign users to combinations of multiple factors"""
    assignments = {}

    for factor_name, factor_levels in factors.items():
        hash_input = f"{user_id}_{factor_name}"
        hash_value = int(hashlib.md5(hash_input.encode()).hexdigest(), 16)
        level_index = hash_value % len(factor_levels)
        assignments[factor_name] = factor_levels[level_index]

    return assignments

# Example: Test both model version and UI layout
factors = {
    'model_version': ['v1', 'v2'],
    'ui_layout': ['layout_a', 'layout_b']
}

assignment = factorial_design_assignment("user_123", factors)
# Result: {'model_version': 'v2', 'ui_layout': 'layout_a'}

```

Best Practices:

1. **Power Analysis:** Calculate required sample sizes before testing
2. **Multiple Testing Correction:** Use Bonferroni or FDR when testing multiple metrics
3. **Segment Analysis:** Analyze results across user segments
4. **Guardrail Metrics:** Monitor for negative effects on key metrics
5. **Gradual Rollout:** Start with small traffic percentages
6. **Documentation:** Maintain experiment registry and results

Common Pitfalls to Avoid:

- Selection bias in user assignment
- Peeking at results too early (optional stopping problem)
- Ignoring network effects
- Not accounting for seasonality
- Testing too many variations simultaneously

Explanation: Effective A/B testing for ML models requires careful experimental design, robust infrastructure, and rigorous statistical analysis. The key is balancing statistical rigor with practical business needs while maintaining the integrity of the experimental process.

Generative AI (GenAI)

20. Explain the architecture and training process of GPT models.

Answer:

GPT Architecture:

1. Transformer Decoder Architecture:

Input Embeddings + Positional Encoding

↓

Decoder Block 1 (Masked Self-Attention + Feed-Forward)

↓

Decoder Block 2

↓

...

↓

Decoder Block N

↓

Layer Normalization

↓

Output Projection (to vocabulary)

↓

Softmax (for next token prediction)

Core Components:

Masked Self-Attention:

python

```
def causal_self_attention(Q, K, V, mask=None):
    """
    Q, K, V: [batch_size, seq_len, d_model]
    Returns attention output with causal masking
    """
    d_k = Q.shape[-1]

    # Compute attention scores
    scores = torch.matmul(Q, K.transpose(-2, -1)) / math.sqrt(d_k)

    # Apply causal mask (prevent looking at future tokens)
    if mask is not None:
        scores = scores.masked_fill(mask == 0, -1e9)

    # Apply softmax
    attention_weights = F.softmax(scores, dim=-1)

    # Apply to values
    output = torch.matmul(attention_weights, V)
    return output
```

Multi-Head Attention:

python

```

class MultiHeadAttention(nn.Module):
    def __init__(self, d_model, num_heads):
        super().__init__()
        self.d_model = d_model
        self.num_heads = num_heads
        self.d_k = d_model // num_heads

        self.W_q = nn.Linear(d_model, d_model)
        self.W_k = nn.Linear(d_model, d_model)
        self.W_v = nn.Linear(d_model, d_model)
        self.W_o = nn.Linear(d_model, d_model)

    def forward(self, x, mask=None):
        batch_size, seq_len = x.shape[:2]

        # Linear projections in batch from d_model => h x d_k
        Q = self.W_q(x).view(batch_size, seq_len, self.num_heads, self.d_k).transpose(1, 2)
        K = self.W_k(x).view(batch_size, seq_len, self.num_heads, self.d_k).transpose(1, 2)
        V = self.W_v(x).view(batch_size, seq_len, self.num_heads, self.d_k).transpose(1, 2)

        # Apply attention
        attention_output = causal_self_attention(Q, K, V, mask)

        # Concatenate heads
        concat_attention = attention_output.transpose(1, 2).contiguous().view(
            batch_size, seq_len, self.d_model
        )

        # Final linear projection
        output = self.W_o(concat_attention)
        return output

```

2. Training Process:

Autoregressive Language Modeling:

python

```

def compute_loss(model, input_ids, labels):
    """
    input_ids: [batch_size, seq_len] - input tokens
    labels: [batch_size, seq_len] - target tokens (shifted input_ids)
    """

    # Forward pass
    logits = model(input_ids) # [batch_size, seq_len, vocab_size]

    # Shift so that tokens < n predict n
    shift_logits = logits[:, :-1, :].contiguous()
    shift_labels = labels[:, 1:].contiguous()

    # Compute cross-entropy loss
    loss_fct = CrossEntropyLoss()
    loss = loss_fct(
        shift_logits.view(-1, shift_logits.size(-1)),
        shift_labels.view(-1)
    )

    return loss

# Training loop
for batch in dataloader:
    input_ids = batch['input_ids']
    labels = input_ids.clone() # Labels are just shifted inputs

    loss = compute_loss(model, input_ids, labels)
    loss.backward()
    optimizer.step()
    optimizer.zero_grad()

```

3. GPT Evolution:

GPT-1 (2018):

- **Parameters:** 117M
- **Innovation:** Demonstrated unsupervised pre-training + supervised fine-tuning
- **Training:** BookCorpus dataset
- **Architecture:** 12 layers, 768 hidden dimensions

GPT-2 (2019):

- **Parameters:** 1.5B (largest variant)
- **Innovation:** Zero-shot task performance, showed scaling benefits

- **Training:** WebText (40GB of internet text)
- **Key insight:** Language models are unsupervised multitask learners

GPT-3 (2020):

- **Parameters:** 175B
- **Innovation:** Few-shot in-context learning
- **Training:** CommonCrawl, WebText2, Books1, Books2, Wikipedia
- **Architecture:** 96 layers, 12,288 hidden dimensions, 96 attention heads

GPT-4 (2023):

- **Parameters:** Estimated 1T+ (not officially disclosed)
- **Innovation:** Multimodal capabilities, improved reasoning
- **Training:** Speculated to include code, books, web text, and images

4. Training Techniques:

Data Preprocessing:

```
python

class GPTDataset(torch.utils.data.Dataset):
    def __init__(self, texts, tokenizer, max_length=1024):
        self.tokenizer = tokenizer
        self.max_length = max_length

        # Tokenize and concatenate all texts
        self.tokens = []
        for text in texts:
            tokens = tokenizer.encode(text, add_special_tokens=True)
            self.tokens.extend(tokens)

    def __len__(self):
        return len(self.tokens) // self.max_length

    def __getitem__(self, idx):
        start_idx = idx * self.max_length
        end_idx = start_idx + self.max_length

        input_ids = torch.tensor(self.tokens[start_idx:end_idx])
        return {'input_ids': input_ids}
```

Gradient Checkpointing for Memory Efficiency:

python

```
class GPTBlock(nn.Module):
    def __init__(self, config):
        super().__init__()
        self.attention = MultiHeadAttention(config.d_model, config.num_heads)
        self.feed_forward = FeedForward(config.d_model, config.d_ff)
        self.ln1 = nn.LayerNorm(config.d_model)
        self.ln2 = nn.LayerNorm(config.d_model)

    def forward(self, x, mask=None):
        # Use gradient checkpointing to save memory
        if self.training:
            return checkpoint(self._forward, x, mask)
        else:
            return self._forward(x, mask)

    def _forward(self, x, mask=None):
        # Pre-norm architecture (used in GPT-2 onwards)
        attn_output = self.attention(self.ln1(x), mask)
        x = x + attn_output

        ff_output = self.feed_forward(self.ln2(x))
        x = x + ff_output

        return x
```

5. Scaling Laws and Optimization:

Chinchilla Scaling Laws:

- Optimal model size depends on compute budget
- For a given compute budget, smaller models trained on more data often outperform larger models trained on less data
- **Rule of thumb:** For every 10x increase in compute, increase model size by $\sim 3.2x$ and data by $\sim 3.2x$

Training Optimizations:

Mixed Precision Training:

python

```
# Use automatic mixed precision for efficiency
scaler = GradScaler()

for batch in dataloader:
    optimizer.zero_grad()

    with autocast():
        outputs = model(batch['input_ids'])
        loss = compute_loss(outputs, batch['labels'])

    scaler.scale(loss).backward()
    scaler.step(optimizer)
    scaler.update()
```

Distributed Training:

```
python

# Data parallel training across multiple GPUs
model = torch.nn.DataParallel(model)

# For very large models, use model parallelism
# Different layers on different GPUs
```

6. Key Training Challenges:

Memory Management:

- **Gradient Checkpointing:** Trade computation for memory
- **Parameter Sharding:** Distribute parameters across devices (ZeRO optimizer)
- **Activation Checkpointing:** Store only subset of activations

Numerical Stability:

- **Layer Normalization:** Stabilize training
- **Gradient Clipping:** Prevent gradient explosion
- **Learning Rate Scheduling:** Warm-up and decay strategies

Data Quality:

- **Deduplication:** Remove repeated content
- **Filtering:** Remove low-quality or harmful content
- **Tokenization:** Handle diverse text formats and languages

Explanation: GPT's success stems from the transformer architecture's ability to model long-range dependencies, combined with massive scale in both model parameters and training data. The autoregressive training objective is simple but powerful, enabling the model to learn rich representations of language that transfer to many downstream tasks.

21. What are the key differences between different generative model architectures (GANs, VAEs, Diffusion Models, Transformers)?

Answer:

Comprehensive Comparison of Generative Models:

1. Generative Adversarial Networks (GANs)

Architecture:

```
python
```

```

class Generator(nn.Module):
    def __init__(self, latent_dim, output_dim):
        super().__init__()
        self.model = nn.Sequential(
            nn.Linear(latent_dim, 128),
            nn.ReLU(),
            nn.Linear(128, 256),
            nn.ReLU(),
            nn.Linear(256, output_dim),
            nn.Tanh() # Output in [-1, 1]
        )

    def forward(self, z):
        return self.model(z)

class Discriminator(nn.Module):
    def __init__(self, input_dim):
        super().__init__()
        self.model = nn.Sequential(
            nn.Linear(input_dim, 256),
            nn.LeakyReLU(0.2),
            nn.Linear(256, 128),
            nn.LeakyReLU(0.2),
            nn.Linear(128, 1),
            nn.Sigmoid() # Probability real/fake
        )

    def forward(self, x):
        return self.model(x)

# Training loop
def train_gan(generator, discriminator, dataloader, num_epochs):
    g_optimizer = torch.optim.Adam(generator.parameters(), lr=0.0002)
    d_optimizer = torch.optim.Adam(discriminator.parameters(), lr=0.0002)
    criterion = nn.BCELoss()

    for epoch in range(num_epochs):
        for real_data in dataloader:
            batch_size = real_data.size(0)

            # Train Discriminator
            d_optimizer.zero_grad()

            # Real data
            real_labels = torch.ones(batch_size, 1)
            real_output = discriminator(real_data)

```

```
d_real_loss = criterion(real_output, real_labels)

# Fake data
noise = torch.randn(batch_size, latent_dim)
fake_data = generator(noise)
fake_labels = torch.zeros(batch_size, 1)
fake_output = discriminator(fake_data.detach())
d_fake_loss = criterion(fake_output, fake_labels)

d_loss = d_real_loss + d_fake_loss
d_loss.backward()
d_optimizer.step()

# Train Generator
g_optimizer.zero_grad()
fake_output = discriminator(fake_data)
g_loss = criterion(fake_output, real_labels) # Want discriminator to think fake is real
g_loss.backward()
g_optimizer.step()
```

Characteristics:

- **Training:** Adversarial (minimax game)
- **Pros:** High-quality samples, fast inference
- **Cons:** Training instability, mode collapse, difficult optimization
- **Best for:** Image generation, style transfer

2. Variational Autoencoders (VAEs)

Architecture:

python

```
class VAE(nn.Module):
    def __init__(self, input_dim, latent_dim):
        super().__init__()

        # Encoder
        self.encoder = nn.Sequential(
            nn.Linear(input_dim, 512),
            nn.ReLU(),
            nn.Linear(512, 256),
            nn.ReLU()
        )

        self.fc_mu = nn.Linear(256, latent_dim)
        self.fc_logvar = nn.Linear(256, latent_dim)

        # Decoder
        self.decoder = nn.Sequential(
            nn.Linear(latent_dim, 256),
            nn.ReLU(),
            nn.Linear(256, 512),
            nn.ReLU(),
            nn.Linear(512, input_dim),
            nn.Sigmoid()
        )

    def encode(self, x):
        h = self.encoder(x)
        mu = self.fc_mu(h)
        logvar = self.fc_logvar(h)
        return mu, logvar

    def reparameterize(self, mu, logvar):
        std = torch.exp(0.5 * logvar)
        eps = torch.randn_like(std)
        return mu + eps * std

    def decode(self, z):
        return self.decoder(z)

    def forward(self, x):
        mu, logvar = self.encode(x)
        z = self.reparameterize(mu, logvar)
        return self.decode(z), mu, logvar

# Loss function
def vae_loss(recon_x, x, mu, logvar):
```

```
# Reconstruction loss
recon_loss = F.binary_cross_entropy(recon_x, x, reduction='sum')

# KL divergence loss
kl_loss = -0.5 * torch.sum(1 + logvar - mu.pow(2) - logvar.exp())

return recon_loss + kl_loss
```

Characteristics:

- **Training:** Maximize ELBO (Evidence Lower Bound)
- **Pros:** Stable training, interpretable latent space, principled probabilistic framework
- **Cons:** Blurry outputs, posterior collapse
- **Best for:** Representation learning, data compression, interpolation

3. Diffusion Models

Forward Process (Noise Addition):

```
python
```

```

def forward_diffusion(x0, t, noise_schedule):
    """Add noise to clean data"""
    noise = torch.randn_like(x0)
    alpha_t = noise_schedule.alpha_cumprod[t]

    #  $x_t = \sqrt{\alpha_t} * x_0 + \sqrt{1 - \alpha_t} * noise$ 
    x_t = torch.sqrt(alpha_t) * x0 + torch.sqrt(1 - alpha_t) * noise
    return x_t, noise

class DiffusionModel(nn.Module):
    def __init__(self, num_timesteps=1000):
        super().__init__()
        self.num_timesteps = num_timesteps

        # U-Net architecture for noise prediction
        self.unet = UNet(
            in_channels=3,
            out_channels=3,
            time_embedding_dim=128
        )

        # Noise schedule
        self.register_buffer('betas', self.cosine_beta_schedule(num_timesteps))
        self.register_buffer('alphas', 1 - self.betas)
        self.register_buffer('alpha_cumprod', torch.cumprod(self.alphas, dim=0))

    def cosine_beta_schedule(self, timesteps, s=0.008):
        """Cosine noise schedule"""
        steps = timesteps + 1
        x = torch.linspace(0, timesteps, steps)
        alphas_cumprod = torch.cos(((x / timesteps) + s) / (1 + s) * torch.pi * 0.5) ** 2
        alphas_cumprod = alphas_cumprod / alphas_cumprod[0]
        betas = 1 - (alphas_cumprod[1:] / alphas_cumprod[:-1])
        return torch.clip(betas, 0.0001, 0.9999)

    def forward(self, x0):
        """Training forward pass"""
        batch_size = x0.shape[0]

        # Sample random timestep
        t = torch.randint(0, self.num_timesteps, (batch_size,))

        # Add noise
        x_t, noise = forward_diffusion(x0, t, self)

        # Predict noise

```

```

predicted_noise = self.unet(x_t, t)

# MSE loss between actual and predicted noise
loss = F.mse_loss(predicted_noise, noise)
return loss

@torch.no_grad()
def sample(self, shape):
    """Reverse diffusion sampling"""
    # Start from pure noise
    x = torch.randn(shape)

    # Reverse process
    for t in reversed(range(self.num_timesteps)):
        t_tensor = torch.full((shape[0],), t, dtype=torch.long)

        # Predict noise
        predicted_noise = self.unet(x, t_tensor)

        # Remove predicted noise
        alpha_t = self.alphas[t]
        alpha_cumprod_t = self.alpha_cumprod[t]

        x = (1 / torch.sqrt(alpha_t)) * (
            x - ((1 - alpha_t) / torch.sqrt(1 - alpha_cumprod_t)) * predicted_noise
        )

        # Add noise (except for last step)
        if t > 0:
            noise = torch.randn_like(x)
            x += torch.sqrt(self.betas[t]) * noise

    return x

```

Characteristics:

- **Training:** Denoise at random timesteps
- **Pros:** High-quality samples, stable training, controllable generation
- **Cons:** Slow inference (many steps), computationally expensive
- **Best for:** Image generation, inpainting, super-resolution

4. Transformer-based Generative Models

Autoregressive Generation:

python

```

class GPTGenerator(nn.Module):
    def __init__(self, vocab_size, d_model, num_heads, num_layers):
        super().__init__()
        self.token_embedding = nn.Embedding(vocab_size, d_model)
        self.position_embedding = nn.Embedding(max_seq_len, d_model)

        self.transformer_blocks = nn.ModuleList([
            TransformerBlock(d_model, num_heads)
            for _ in range(num_layers)
        ])

        self.ln_f = nn.LayerNorm(d_model)
        self.head = nn.Linear(d_model, vocab_size, bias=False)

    def forward(self, idx, targets=None):
        b, t = idx.size()
        pos = torch.arange(0, t, dtype=torch.long).unsqueeze(0) # shape (1, t)

        # Forward pass
        tok_emb = self.token_embedding(idx) # token embeddings
        pos_emb = self.position_embedding(pos) # position embeddings
        x = tok_emb + pos_emb

        for block in self.transformer_blocks:
            x = block(x)

        x = self.ln_f(x)
        logits = self.head(x)

        loss = None
        if targets is not None:
            loss = F.cross_entropy(logits.view(-1, logits.size(-1)), targets.view(-1))

        return logits, loss

    @torch.no_grad()
    def generate(self, idx, max_new_tokens, temperature=1.0, top_k=None):
        """Generate new tokens autoregressively"""
        for _ in range(max_new_tokens):
            # Crop context if needed
            idx_cond = idx if idx.size(1) <= max_seq_len else idx[:, -max_seq_len:]

            # Forward pass
            logits, _ = self(idx_cond)
            logits = logits[:, -1, :] / temperature # Last token, scale by temperature

```

```
# Optionally crop probabilities to only the top k options
```

```
if top_k is not None:
```

```
    v, _ = torch.topk(logits, min(top_k, logits.size(-1)))
```

```
    logits[logits < v[:, [-1]]] = -float('Inf')
```

```
probs = F.softmax(logits, dim=-1)
```

```
idx_next = torch.multinomial(probs, num_samples=1)
```

```
idx = torch.cat((idx, idx_next), dim=1)
```

```
return idx
```

Non-Autoregressive Generation (BERT-like):

python

```
class MaskedLanguageModel(nn.Module):
```

```
    def __init__(self, vocab_size, d_model, num_heads, num_layers):
```

```
        super().__init__()
```

```
        # Similar architecture to GPT but with bidirectional attention
```

```
        # Used for masked token prediction
```

```
@torch.no_grad()
```

```
def iterative_generation(self, length, num_iterations=10):
```

```
    """Generate by iteratively predicting masked tokens"""
```

```
    # Start with all mask tokens
```

```
    tokens = torch.full((1, length), mask_token_id)
```

```
    for _ in range(num_iterations):
```

```
        # Forward pass
```

```
        logits = self(tokens)
```

```
        # Sample predictions for masked positions
```

```
        probs = F.softmax(logits, dim=-1)
```

```
        predicted_tokens = torch.multinomial(probs.view(-1, vocab_size), 1).view(1, -1)
```

```
        # Update some percentage of tokens
```

```
        mask = torch.rand(1, length) < 0.15 # Keep 15% masked
```

```
        tokens = torch.where(mask, mask_token_id, predicted_tokens)
```

```
    return tokens
```

Architectural Comparison Table

Aspect	GANs	VAEs	Diffusion	Transformers
Training	Adversarial	Variational inference	Denoising	Maximum likelihood
Inference Speed	Fast (1 pass)	Fast (1 pass)	Slow (many steps)	Slow (autoregressive)
Sample Quality	High	Medium	Very High	High (text)
Training Stability	Poor	Good	Good	Good
Controllability	Medium	High	High	High
Latent Space	Implicit	Explicit	Implicit	Discrete (tokens)
Mode Coverage	Poor (mode collapse)	Good	Excellent	Good

Use Case Recommendations

GANs:

- **Best for:** High-resolution image generation, style transfer, real-time applications
- **Examples:** StyleGAN for faces, CycleGAN for domain transfer
- **Avoid when:** Need stable training or diverse outputs

VAEs:

- **Best for:** Data compression, representation learning, controlled generation
- **Examples:** Molecule generation, music composition, anomaly detection
- **Avoid when:** Need sharp, high-quality outputs

Diffusion Models:

- **Best for:** High-quality image generation, inpainting, super-resolution
- **Examples:** DALL-E 2, Midjourney, Stable Diffusion
- **Avoid when:** Need fast inference or limited compute

Transformers:

- **Best for:** Text generation, code generation, multimodal tasks
- **Examples:** GPT for text, Codex for code, DALL-E for text-to-image
- **Avoid when:** Need continuous latent spaces or non-sequential data

Hybrid Approaches

VQ-VAE (Vector Quantized VAE):

- Combines discrete latent codes with autoencoder framework
- Used in DALL-E, enables transformer-based image generation

Diffusion Transformers:

- Replace U-Net with transformer architecture in diffusion models
- Better scalability and performance on certain tasks

GAN + VAE (VAE-GAN):

- Use VAE encoder with GAN discriminator
- Combines reconstruction loss with adversarial training

Explanation: Each generative model architecture has distinct strengths and weaknesses. The choice depends on your specific requirements for quality, speed, controllability, and training stability. Modern systems often combine multiple approaches to leverage their complementary strengths.